

Can RL Teach Long-Horizon Reasoning to LLMs? Expressiveness Is Key

Tianle Wang¹ Zhaoyang Wang² Guangchen Lan¹ Xinpeng Wei³
Sipeng Zhang⁴ Guanwen Qiu¹ Abulhair Saparov¹

¹Purdue University ²UNC Chapel Hill ³Georgia Tech ⁴UC San Diego
wang7327@purdue.edu

Abstract

Reinforcement learning (RL) has been applied to improve large language model (LLM) reasoning, yet the systematic study of how training scales with task difficulty has been hampered by the lack of controlled, scalable environments. We introduce SCALELOGIC, a synthetic logical reasoning framework that offers independent control over two axes of difficulty: the depth of the required proof planning (i.e., the *horizon*) and the expressiveness of the underlying logic. Our proposed framework supports a wide range of logics: from simple implication-only logic (“if-then”) towards more expressive first-order reasoning with conjunction (“and”), disjunction (“or”), negation (“not”), and universal quantification (“for all”). Using this framework, we show that the RL training compute T follows a power law with respect to reasoning depth D — $T \propto D^\gamma$, $R^2 > 0.99$ —and that the scaling exponent γ increases monotonically with logical expressiveness, from 1.04 to 2.60. On downstream mathematics and general reasoning benchmarks, more expressive training settings yield both larger performance gains (up to +10.66 points) and more compute-efficient transfer compared to less expressive settings, demonstrating that *what* a model is trained on, not just *how much* it is trained, shapes downstream transfer. We further show that the power-law relationship holds across multiple RL methods, and curriculum-based training substantially improves scaling efficiency.

1 Introduction

Recent advances in reinforcement learning (RL) for large language model (LLM) reasoning have shown that training on verifiable domains such as mathematics and coding can elicit enhanced reasoning behavior and improve benchmark performance [Guo et al., 2025a, Jaech et al., 2024]. However, these gains do not yet translate into robust long-horizon reasoning, as the same model’s performance can degrade sharply once tasks require many sequential reasoning steps, even when the corresponding subproblems are individually manageable [Rameshkumar et al., 2025, Motwani et al., 2025, Lu et al., 2025, Zhou et al., 2025].

A key limitation is that scalable long-horizon reasoning training requires three properties simultaneously: exact verifiability, fine-grained control over reasoning difficulty, and data available at scale that supports systematic analysis. The domains currently driving RL progress, such as mathematics and coding, often satisfy the first property but not the latter two: high-quality problems are expensive to curate and offer only limited control over horizon and difficulty [Hendrycks et al., 2021, Jain et al., 2024]. Existing alternatives, including synthetic tasks [Xie et al., 2025, Liu et al., 2025a] and self-evolving pipelines [Zhao et al., 2025, Huang et al., 2025], reduce the cost of data collection through automatic generation and verification, but often do not provide a sufficiently clean and interpretable set of control axes for analyzing scaling behavior. As a result, we still lack a controlled training setting for studying how RL scales with increasingly difficult long-horizon reasoning problems. Table 1 summarizes how existing data sources fall short on at least one of these axes.

Table 1: Comparison of training data sources for RL post-training of LLM reasoning. Here, ✓ denotes direct support, ◦ denotes coarse or indirect control, and × denotes no direct control; e.g., SAT can vary the number of variables and clauses, but only indirectly controls the reasoning horizon.

Data Source	Verifiable	Scalable	Controllable Horizon	Controllable Expressiveness
Math / Code [e.g., Hendrycks et al., 2021]	✓	×	×	×
Game-based [e.g., Wu et al., 2024]	◦	✓	×	×
Self-Evolving [e.g., Zhao et al., 2025]	✓	✓	×	×
Knights and Knaves [Xie et al., 2025]	✓	✓	◦	×
SAT [Liu et al., 2025a]	✓	✓	◦	×
G1 [Guo et al., 2025b]	✓	✓	◦	×
Puzzle Suites [e.g., Liu et al., 2025b]	✓	✓	◦	×
SCALELOGIC	✓	✓	✓	✓

To address this gap, we propose SCALELOGIC, a synthetic logical reasoning environment with explicit control over two key dimensions of difficulty: the depth of the required proof planning (i.e., the *horizon*) and the logical expressiveness of the reasoning problems. As illustrated in Figure 1, in each example, the model is given a set of facts and a handful of candidate conclusions. The model is then asked to identify which candidate conclusion is logically derivable from those facts, yielding an exact and easily verifiable multiple-choice objective. To find the correct multiple-choice option, the model must search for a proof of each candidate conclusion. Our environment requires it to expend comparable reasoning effort to determine the provability of each candidate, which substantially limits the model’s ability to exploit simple heuristics or shortcuts during reasoning. Beyond scaling the depth of the proof tree, our environment allows us to vary the expressive structure of the underlying logic, including the logical operators and structural properties involved in each example. Combined with automatic low-cost generation and verification, this provides a controlled testbed for systematically studying RL scaling in long-horizon reasoning.

Experiments in this controlled environment show that over the observed depth range, the RL training steps T required to reach 90% validation accuracy follow a power law in the proof-tree depth D ($T \propto D^\gamma$, $R^2 > 0.99$), where the scaling exponent γ increases monotonically with logical expressiveness, from 1.04 to 2.60. This indicates that more logically expressive settings require disproportionately more training to solve problems of the same depth. We also find that the power-law relationship holds across different RL methods, and these dynamics depend strongly on the training distribution: a carefully designed curriculum improves training efficiency and stability.

We also demonstrate that training a model in our synthetic reasoning environment improves performance on downstream real-world mathematics and general reasoning benchmarks, with the most expressive setting improving the mean accuracy across eight benchmarks by up to +10.66 percentage points over the base model. Critically, training on highly expressive data yields a monotonically increasing downstream performance curve, while less expressive settings plateau early. This indicates that *what* the model is trained on, not just *how much* it is trained, shapes downstream transfer. Taken together, these results suggest that the logical expressiveness of the training data influences not only how RL scales on synthetic reasoning tasks, but also the extent to which such training can provide transferable improvement in reasoning performance for real-world applications.

In summary, we introduce a scalable synthetic reasoning framework for studying and improving long-horizon reasoning in RL post-training. Our main contributions are as follows:

- We propose SCALELOGIC, a controlled and scalable synthetic logical reasoning testbed for RL post-training, with exact verifiability, low-cost automatic generation, and explicit control over reasoning horizon and logical expressiveness.
- We show that RL training effort follows a power law with respect to proof tree depth, with the scaling exponent γ increasing monotonically with logical expressiveness, from 1.04 to 2.60. The power-law relationship holds across multiple RL methods, and we further show that curriculum training improves scaling efficiency.
- We show that synthetic reasoning training improves downstream reasoning performance by up to +10.66 points, and these gains depend strongly on the logical expressiveness of the training environment, with more expressive settings yielding larger and more compute-efficient transfer.

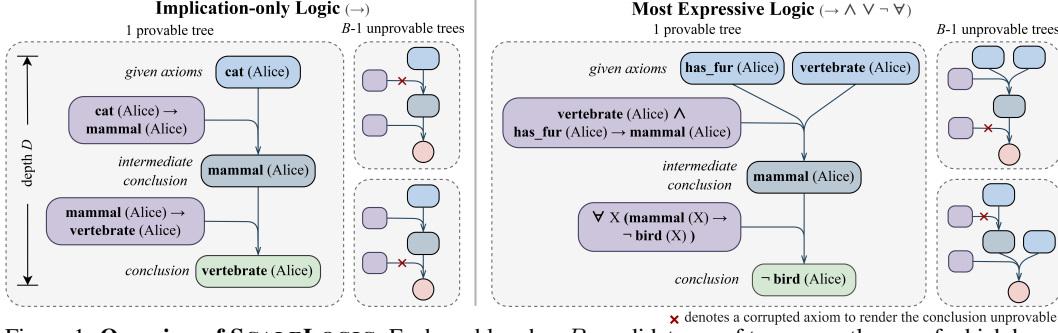


Figure 1: **Overview of SCALELOGIC.** Each problem has B candidate proof trees, exactly one of which has a provable conclusion; the others are made unprovable by corrupting one axiom. The depth D controls proof depth. **Left:** Implication-only reasoning. **Right:** The most expressive logic setting (referred to as + *Quantification* in Section 3.2) combines conjunction, disjunction, negation, and universal quantification.

2 Related Work

Long-horizon Reasoning Limitations. Recent work reveals that LLMs often exhibit sharp performance degradation as the reasoning horizon increases. Rameshkumar et al. [2025] show that reasoning models can perform well on graph-based reasoning tasks within a limited complexity regime, but their performance drops abruptly once the reasoning horizon is sufficiently large. SeqBench [Ramezani et al., 2025] and GSM-Infinite [Zhou et al., 2025] also report steep performance degradation with exponential and sigmoid-like decay patterns. Further studies, such as R-Horizon [Lu et al., 2025] and h1 [Motwani et al., 2025], compose individually solvable mathematical problems into multi-step dependency chains and investigate whether RL training can mitigate long-horizon failures. Building on this line of work, we characterize how RL training behaves as reasoning structures are systematically scaled, and how these scaling dynamics shape downstream transfer.

Scaling in LLMs. Early scaling-law studies found regular power laws relating pre-training performance to model scale, data volume, and training compute [Kaplan et al., 2020, Henighan et al., 2020, Hoffmann et al., 2022]. Later, test-time scaling emerged to improve reasoning by allocating additional computation during decoding [Wei et al., 2022, Wang et al., 2023, Yao et al., 2023, Muennighoff et al., 2025]. Recent work has extended scaling-law analysis to RL post-training, showing regular scaling behavior with model scale, data, and compute [Khatri et al., 2025, Tan et al., 2025]. However, existing RL scaling studies primarily vary the volume of training data, while providing limited control over the reasoning complexity of individual problems. In contrast, our work enables a cleaner analysis through explicit and interpretable control of reasoning complexity.

RL for LLM Reasoning. RL with verifiable rewards (RLVR) has become a promising paradigm for reasoning-oriented post-training [Luong et al., 2024, Guo et al., 2025a]. With practical optimizers such as GRPO [Shao et al., 2024] and later variants [Yu et al., 2025, Zheng et al., 2025], it has enabled large-scale RL training and long chain-of-thought reasoning [Jaech et al., 2024, Guo et al., 2025a]. However, most existing RL reasoning work centers on mathematics and programming, where high-quality training problems are limited, often depend on human-curated solutions or test cases, and offer only coarse difficulty control [Liu et al., 2025a]. As a result, RL performance is limited by the collected data, making sustained improvement difficult as training scales. In contrast, SCALELOGIC offers explicit complexity control, verifiable solutions, and unlimited low-cost data generation, enabling a cleaner and more scalable framework for reasoning-oriented RL.

Synthetic Data for Post-training. Synthetic data is a natural fit for RLVR, motivating recent work on automatically generated reasoning problems with verifiable rewards. Early efforts studied task-specific synthetic settings for logical reasoning, such as Knights and Knaves [Xie et al., 2025, Lin et al., 2025]. More recent work shifted toward task families with difficulty control such as SAT [Liu et al., 2025a], graph reasoning tasks such as G1 [Guo et al., 2025b], and benchmark-style synthetic reasoning suites [Liu et al., 2025b, Chen et al., 2025, Stojanovski et al., 2025, He et al., 2026]. While these studies show the promise of synthetic data for post-training, their tasks are typically worst-case NP-hard search problems (e.g., SAT and Hamiltonian path in G1) with limited control over underlying expressiveness, making it difficult to isolate how task complexity affects RL training compute and downstream reasoning. In contrast, SCALELOGIC independently controls proof depth and logical expressiveness with each instance oracle-solvable in time linear in the proof size.

3 Method

We present SCALELOGIC, a framework for generating synthetic logical reasoning problems with fine-grained control over task difficulty. Section 3.1 describes how reasoning problems are constructed, Section 3.2 explains how the difficulty of the generated problems is systematically controlled through logical expressiveness, and Section 3.3 describes the reinforcement learning setup for post-training.

3.1 Generation of Logical Reasoning Problems

Our training environment is built on a pipeline for generating synthetic logical reasoning problems. We refer to a grounded predicate applied to a specific entity, possibly negated, such as “Alice is a cat” (written in logic as `cat(Alice)`) or “Alice is not a cat” (`¬cat(Alice)`), as a *literal*. Each instance consists of a collection of axioms, which include literals such as “Alice is a cat” (`cat(Alice)`) and rules such as “If Alice is a cat, then Alice is a mammal” (`cat(Alice) → mammal(Alice)`), which together determine the set of conclusions that can be logically derived, such as “Alice is a mammal” (`mammal(Alice)`). The underlying task is to identify, among a set of candidate conclusions (each a single literal), which one is logically derivable from the given axioms.

Each instance is presented as a single-answer multiple-choice problem. To construct it, we first sample B literals, each serving as the root of a proof tree. Starting from each root, we recursively expand the tree by adding child literals to its leaves. Each parent node, along with its set of children, defines a proof step: the children serve as the premises, while the parent is the conclusion. The conclusion of one proof step may in turn serve as a premise for another step higher in the tree. Thus, expanding a leaf amounts to generating its supporting premises, which become new leaves of the proof tree. We repeat this process until a target proof depth D is reached, at which point the leaves are treated as the literal axioms from which the proof begins (see Algorithm 1). This procedure generates proofs “backwards”: it starts from the conclusion and progressively constructs the premises needed to derive it. To avoid introducing alternative derivations, each expansion adds premise literals with fresh predicates that do not appear elsewhere in any proof trees. This ensures that every node has a unique derivation from the axioms, corresponding to the subtree rooted at that node.¹ Applying the procedure to the B roots yields one proof tree for each candidate conclusion.

We keep one proof tree intact so that its root conclusion remains derivable. For each of the remaining $B - 1$ candidates, we uniformly sample one axiom from the proof and corrupt it in one of two ways: (i) *removing* the axiom, or (ii) *flipping* the polarity of one literal within the axiom, e.g., changing “Alice is a cat” (`cat(Alice)`) to “Alice is not a cat” (`¬cat(Alice)`). Note that option (ii) is available only when negation is included in the underlying logic; it is disabled in less expressive settings that do not support negation (see Appendix D). The uniform sampling process prevents the model from exploiting positional shortcuts. Since our backward construction yields a unique proof for each candidate, corrupting a single axiom severs the proof path to the root and makes the corresponding conclusion non-derivable. We additionally insert a small number of distracting rules to increase local ambiguity without enabling any new valid derivations; their construction is detailed in Appendix C.2.

This construction yields two interpretable structural variables, B and D , that directly govern instance complexity. Increasing B introduces more plausible candidate conclusions that the model must distinguish, while increasing D requires reasoning over longer proof chains. At the same time, the formulation allows exact verification of the final answer without supervising the entire proof, making it a natural fit for reinforcement learning with verifiable rewards.

After construction, we convert the axioms and candidate conclusions into natural language via predefined templates, instantiated with randomly sampled entity names and fake predicate words (see Appendix C.3). An illustrative example is provided below and algorithms are given in Appendix C.

Example: Natural Language Instance

Suppose we have the following facts:

Alice has fur. Alice is a vertebrate. If Alice has fur and is a vertebrate, then Alice is a mammal.

If anyone is a mammal, then they are not a bird. ...

Here are some candidate statements: (A) Alice is not a bird. (B) Alice is cold-blooded. (C) ...

Which one of the above statements can be logically derived from the given facts?

Answer: Alice is not a bird.

¹ Similar to the generation procedure in Opedal et al. [2025] which also guarantees the uniqueness of generated proofs.

3.2 Control of Logical Expressiveness

Beyond depth and candidate count, we vary the expressive power of the underlying logic while preserving the same task format. We consider a hierarchy of five settings, each a strict superset of the previous, so that any increment in reasoning difficulty can be cleanly attributed to the newly introduced logical features. We describe each level in turn, starting with the simplest.

Implication-only. We start with a simple logic that contains only the implication operator (i.e., “if-then”, written $\rightarrow$). Each axiom is either a grounded literal, such as “Alice is a cat” (`cat(Alice)`), or a simple implication rule such as “If Alice is a cat, then Alice is a mammal” (`cat(Alice)  $\rightarrow$  mammal(Alice)`).² A statement is considered valid if it can be derived from the grounded literals by repeatedly applying rules whose antecedents are satisfied.

+ Conjunction. We extend the simplest logic with the conjunction operator (i.e., “and”, written $\wedge$), which allows a rule to depend on multiple premises simultaneously.³ Under this logic, a rule may take the form of a conjunction of grounded literals entailing a single conclusion, such as “If Alice is a vertebrate and has fur, then Alice is a mammal” (`vertebrate(Alice)  $\wedge$  has_fur(Alice)  $\rightarrow$  mammal(Alice)`). During inference, the model must coordinate multiple satisfied supporting literals before applying each rule, rather than relying on single-premise inference.

+ Negation. We further extend the logic with the negation operator (i.e., “not”, written $\neg$), which allows premises and conclusions to involve negated literals. Under this logic, rules may condition on the absence of a property or derive such an absence as a consequence of other literals, such as “If Alice is a mammal, then Alice is not a bird” (`mammal(Alice)  $\rightarrow$   $\neg$ bird(Alice)`). With negation available, each literal in a proof tree has a well-defined negated counterpart, so the model must track not only whether a predicate has been established but also its polarity.

+ Disjunction. We further extend the logic with the disjunction operator (i.e., “or”, written $\vee$), which allows a rule to produce multiple possible consequents from the same premises. Under this logic, a rule may take the form of the antecedent entailing a disjunction of grounded literals, such as “If Alice is a pet, then Alice is a cat or a dog” (`pet(Alice)  $\rightarrow$  cat(Alice)  $\vee$  dog(Alice)`). With disjunction, the model must reason over multiple possible conclusions, determining which alternatives are eliminated and which support the target statement.

+ Quantification. The most expressive logic we consider extends the earlier logic from purely propositional reasoning towards first-order reasoning, through *universal quantification* (i.e., “for all”, written $\forall$), which enables the definition of rules that apply to *any* entity rather than a specific one. For example, “Anyone who is a cat is a mammal” ($\forall X(\text{cat}(X) \rightarrow \text{mammal}(X))$). At inference time, applying such quantified rules requires the model to instantiate them with a concrete entity in the current context, and to verify that the instantiated antecedents hold before deriving the consequents.

We provide further details on the construction and use of these logical features in Appendix D.

3.3 Reinforcement Learning Framework

Our primary RL algorithm is DAPO [Yu et al., 2025], an extension of Group Relative Policy Optimization (GRPO) [Shao et al., 2024]. We first describe the GRPO objective on which our recipe is built. For each prompt q , we sample a group of G completions $\{o_i\}_{i=1}^G$ from the policy model. The GRPO objective is

$$\mathcal{L}_{\text{GRPO}}(\theta) = \mathbb{E}_{q, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min \left(r_{i,t}(\theta) \hat{A}_i, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) \right],$$

where $r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}$, $\hat{A}_i = \frac{R_i - \text{mean}(\{R_j\}_{j=1}^G)}{\text{std}(\{R_j\}_{j=1}^G)}$.

Here, $r_{i,t}(\theta)$ is the token-level policy ratio, and $\hat{A}_i$ is the group-normalized advantage computed from the scalar completion rewards $\{R_i\}_{i=1}^G$. On top of this objective, our DAPO recipe employs the dynamic sampling and clip-higher strategies from Yu et al. [2025] to improve training efficiency.

²This logic is also known as *implicational propositional logic* or *implicational propositional calculus*.

³We only permit conjunctions in the antecedent of if-then statements, since $A \rightarrow B \wedge C$ is equivalent to $A \rightarrow B$ and $A \rightarrow C$. Similarly, we only permit disjunctions in the consequent.

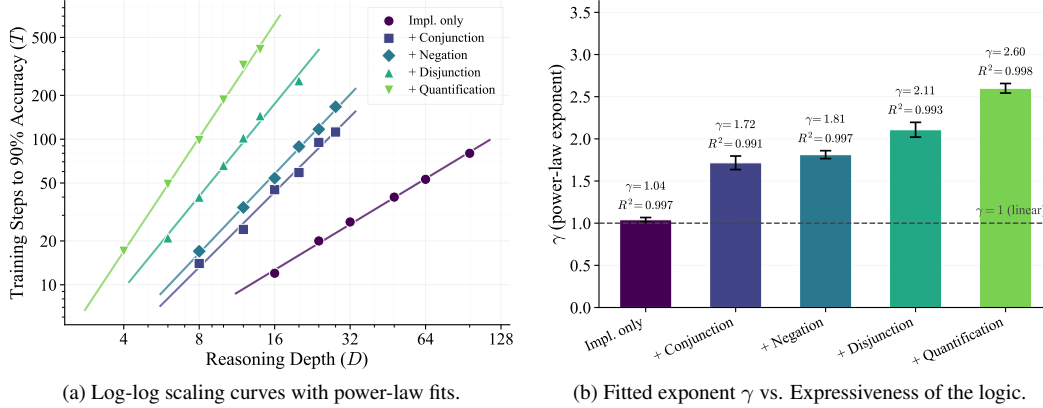


Figure 2: **Training cost scales as a power law with reasoning depth, with exponent γ governed by logical expressiveness.** (a) Scatter points show the training steps T required to reach the 90% accuracy threshold as a function of reasoning depth D . Solid lines show power-law fits $T \propto D^\gamma$. (b) Fitted exponent γ increases monotonically when expressiveness increases from *Implication-only* ($\gamma = 1.04$) to + *Quantification* ($\gamma = 2.60$). Error bars denote ± 1 standard error of the fitted exponent.

For reward design, we adopt a simple and verifiable binary reward, following common practice in reasoning-oriented RL [Guo et al., 2025a]. Specifically, we require the model to place its final answer within `<answer> . . . </answer>`. During evaluation, the verifier extracts the predicted answer from this span and compares it with the ground-truth answer via exact match. If the output violates the required format or the extracted answer does not match the ground truth, we set $R_i = 0$; otherwise, $R_i = 1$. Full prompt templates are provided in Appendix G.

4 Experiments

In this section, we aim to answer the following research questions.

- RQ1 Scaling with complexity** (§4.2). How does the cost of training a model with RL to reach a target accuracy scale with reasoning depth and logical expressiveness?
- RQ2 Downstream transfer** (§4.3). Does training on synthetic reasoning tasks improve performance on real-world benchmarks, and how does expressiveness affect transfer?
- RQ3 Training distribution** (§4.4). How does the training distribution affect scaling efficiency?
- RQ4 Generalization across RL algorithms** (§4.5). Is the observed scaling behavior algorithm-specific, or does it reflect a broader phenomenon across RL methods?
- RQ5 OOD generalization** (§4.6). Does training generalize to more difficult (unseen) depths?

4.1 Experimental Setup

Models and implementation. We perform RL post-training on the non-thinking version of Qwen3-4B [Yang et al., 2025] using the ver1 library [Sheng et al., 2024]. To assess cross-scale generality, we replicate a subset of experiments on Qwen3-8B; results are reported in Appendix I.5. Full implementation details and hyperparameters are provided in Appendix F.

Task configurations. We vary task difficulty along two axes: *reasoning depth* and *logical expressiveness*. Unless otherwise specified, we fix the number of candidates to $B = 4$ throughout, and study the effect of varying B separately in Appendix I.1. For each target depth D , we construct a training set by uniformly sampling instances from depths $\{1, \dots, D\}$, choosing a dataset size (typically 100,000) sufficient for convergence within a single epoch, with each instance seen only once. Evaluation is performed on a held-out validation set of 1,000 instances from the same configuration. We study five levels of logical expressiveness, ranging from implication-only reasoning towards increasingly expressive first-order reasoning, as summarized in Table 2. To confirm task difficulty, we evaluate six frontier LLMs on SCALELOGIC; all models degrade substantially at large depths (see Appendix K).

Training protocol and compute metric. For each configuration, we train a separate model and evaluate it on the corresponding held-out validation set after every RL training step. Our primary measure of training compute, denoted by T , is the number of RL training steps required to reach a

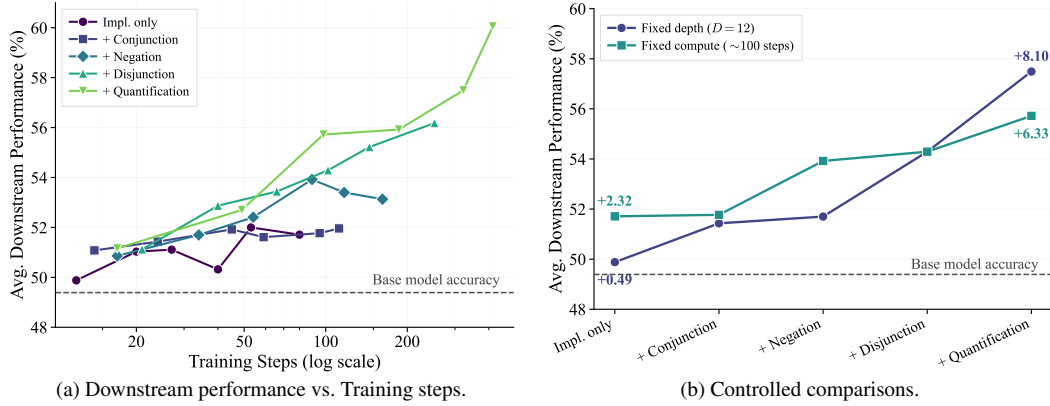


Figure 3: **Downstream transfer from synthetic reasoning training.** (a) All settings outperform the base model, with richer logical settings producing larger and more sustained gains. (b) Average downstream performance across logical settings under *fixed depth* ($D = 12$) and *fixed compute* (~ 100 steps). Both controls exhibit the same monotone trend: More expressive training settings yield stronger downstream performance.

held-out Pass@1 threshold of $\mu = 90\%$. We use this metric throughout the main text and discuss additional compute-related quantities, including generated tokens and FLOPs, in Appendix H.3.

Downstream benchmarks. To evaluate transfer beyond the synthetic environment, we test the trained models on a diverse set of downstream reasoning benchmarks, including AIME 2024 and 2025, AMC 2023, MATH-500 [Hendrycks et al., 2021], Minerva [Lewkowycz et al., 2022], OlympiadBench text-only subset [He et al., 2024], GPQA-Diamond [Rein et al., 2023], and a STEM subset of MMLU-Pro [Wang et al., 2024]. Our main downstream metric is the mean accuracy across the eight benchmarks, with each benchmark evaluated using Avg@8, the average accuracy over eight independently sampled completions per problem. Detailed results for each benchmark are reported in the appendix I.2.

4.2 Power-Law Scaling with Depth and Expressiveness

We first study how training effort scales with reasoning depth across different levels of logical expressiveness. As shown in Figure 2a, over the observed depth range, training compute T follows a power law with respect to the reasoning depth D , namely $T = a \cdot D^\gamma$, across all five expressiveness levels, with $R^2 > 0.99$ in every case. We fit this relationship via ordinary least squares in log-log space. The power-law model consistently outperforms an exponential fit across all settings ($\Delta\text{AIC} \geq +7.1$; see Appendix H.1), suggesting that training cost grows polynomially rather than exponentially with reasoning depth in the studied regime.

While all settings follow a power-law form, the fitted exponent γ increases monotonically with logical expressiveness (Figure 2b), from $\gamma = 1.04 \pm 0.03$ for *Implication-only* to $\gamma = 2.60 \pm 0.06$ for *+ Quantification*. Thus, doubling reasoning depth increases training cost by roughly $2\times$ in the simplest setting, but about $6\times$ in the most expressive setting. The near-linear exponent for *Implication-only* suggests that, when the logical structure is sufficiently simple, each additional unit of depth imposes an approximately constant marginal learning cost: the model primarily learns to “chain one more step.” By contrast, richer logical operators introduce additional combinatorial structure, such as jointly verifying multiple premises under conjunction, which causes depth-related costs to compound more rapidly. Notably, *+ Conjunction* ($\gamma = 1.72 \pm 0.08$) and *+ Negation* ($\gamma = 1.81 \pm 0.05$) exhibit the smallest gap, with partially overlapping standard-error ranges. This is consistent with negation not introducing additional combinatorial structure, but instead mainly requiring the model to track literal polarity. This may also partly reflect the absence of inference rules such as proof by contradiction. Appendix H shows that these patterns are robust to a range of design and analysis choices.

4.3 Downstream Transfer from Synthetic Reasoning Tasks

We next study whether learning on synthetic reasoning tasks transfers to real-world downstream benchmarks. Figure 3a reports average downstream performance as a function of RL training steps across all five expressiveness levels. All settings improve over the base model (49.39%), confirming that the reasoning skills learned in our synthetic environment are not confined to the

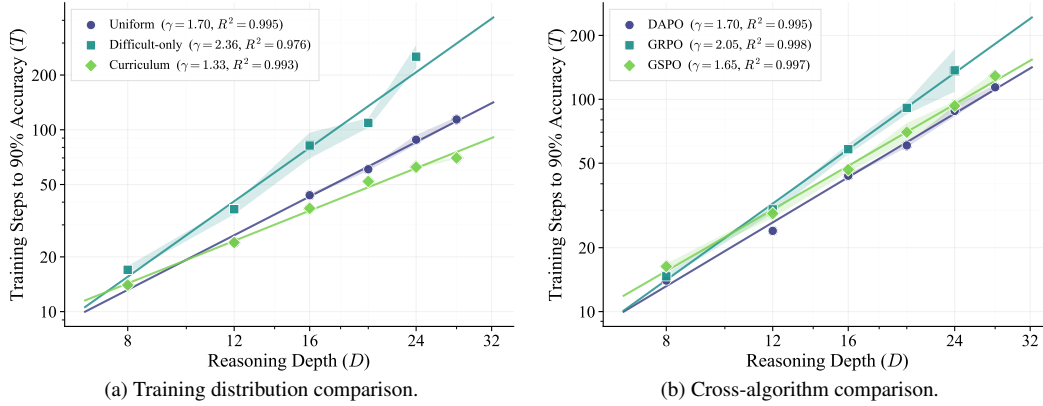


Figure 4: **Effect of training distribution and RL algorithm on scaling efficiency in the + Conjunction setting.** Each curve aggregates three independent seeds; shading denotes ± 1 standard deviation in log-space across seeds. (a) Curriculum training yields the lowest exponent ($\gamma = 1.33$), whereas difficult-only training yields the highest ($\gamma = 2.36$). (b) All RL algorithms follow power-law scaling ($R^2 > 0.99$), with exponents from 1.65 to 2.05.

training domain. However, the size and persistence of these gains differ markedly depending on the logical expressiveness of the training examples. Simpler settings (*Implication-only* and + *Conjunction*) plateau early around 52%, whereas the + *Quantification* setting continues to improve throughout training, reaching 60.05% at 414 steps—an absolute gain of 10.66 percentage points.

To disentangle the contributions of expressiveness and training compute, Figure 3b compares two controlled conditions across the five expressiveness levels. The *fixed depth* line holds the target reasoning depth at $D = 12$, isolating the effect of expressiveness at a comparable task horizon.⁴ The *fixed compute* line selects the checkpoint closest to 100 training steps for each setting, controlling for training budget. Under fixed depth, downstream gains increase from +0.49 points for *Implication-only* to +8.10 points for + *Quantification*, indicating a strong association between logical expressiveness and downstream transfer. Under fixed compute, gains follow the same monotone trend, rising from +2.32 to +6.33 points. Together, these controls suggest that downstream reasoning transfer depends strongly on *what* the model is trained on, not merely on *how much* training it receives.

4.4 Effect of Training Distribution on Scaling Efficiency

We next study how the distribution of training difficulty affects the efficiency and stability of RL scaling. Under the + *Conjunction* setting, we compare three strategies for a target depth D : **uniform** sampling over depths $\{1, \dots, D\}$ as our default setting, **curriculum** sampling that gradually increases the depth during training, and **difficult-only** training that uses only depth- D instances. We run each setting with three seeds per depth. The implementation details are provided in Appendix F.2.

As shown in Figure 4a, the training distribution has a major effect on scaling efficiency. All three strategies maintain power-law scaling, but the fitted exponents differ substantially: curriculum training achieves the lowest exponent ($\gamma = 1.33$), followed by uniform training ($\gamma = 1.70$), while difficult-only training scales most steeply ($\gamma = 2.36$) and exhibits the largest variance. Appendix I.3 confirms the same benefit under + *Quantification*, lowering γ from 2.60 with uniform training to 2.30 with curriculum training. To better understand this effect, we analyze training dynamics in Appendix I.4. Under curriculum training, long chain-of-thought behavior emerges at broadly similar stages across depths; under uniform or difficult-only training, however, its emergence is increasingly delayed and often irregular (e.g., note the large gap between depths 20 and 24). These patterns suggest that curriculum training improves scaling efficiency by accelerating long-CoT emergence across depths, possibly because shallower instances provide a smoother bootstrap for learning useful reasoning patterns. We leave a thorough investigation of this mechanism to future work.

4.5 Cross-Algorithm Robustness

To assess whether the observed scaling behavior is specific to DAPO or reflects a broader property of RL training on these tasks, we repeat the scaling experiment in the + *Conjunction* setting with two

⁴For the *Implication-only* setting, we use $D = 16$ because $D = 12$ is too easy for training.

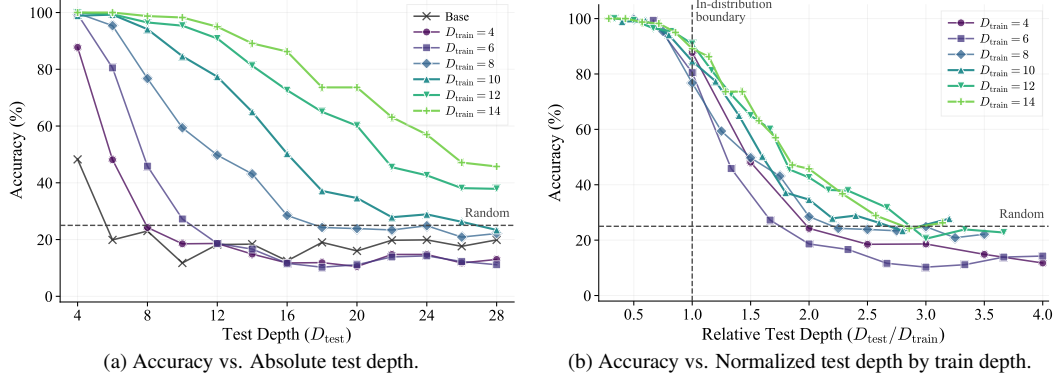


Figure 5: **Out-of-distribution generalization across reasoning depths.** (a) Increasing the training depth consistently extends the range of solvable test depths. (b) OOD generalization remains bounded: even the models trained at the largest depths fall to random at $D_{\text{test}}/D_{\text{train}} \approx 3$.

additional algorithms: base GRPO [Shao et al., 2024] (without DAPO extension) and GSPO [Zheng et al., 2025], a recent sequence-level policy optimization variant. We run each algorithm with three seeds per depth. As shown in Figure 4b, all three algorithms exhibit clear power-law scaling ($R^2 > 0.99$), confirming that the relationship $T \propto D^\gamma$ is not an artifact of a particular optimization method. DAPO and GSPO have similar fitted exponents ($\gamma = 1.70$ and 1.65 , respectively), while GRPO exhibits a steeper scaling exponent ($\gamma = 2.05$) and greater variance at larger depths, suggesting lower sample efficiency in the long-horizon regime.

4.6 Out-of-Distribution Generalization

We examine whether training at a target reasoning depth generalizes to higher unseen depths by evaluating models trained with different depths under + *Quantification*. Figure 5a shows that larger training depths consistently extend the range over which models maintain strong performance: shallow-depth models degrade rapidly, while deeper-trained models preserve higher accuracy on harder evaluations. This suggests that training expands the model’s effective reasoning horizon rather than merely improving performance at the target depth. However, this generalization remains bounded. Even the models trained at the largest depths eventually degrade at sufficiently large evaluation depths. After normalizing test depth by training depth (Figure 5b), the two deepest models ($D_{\text{train}} = 12, 14$) collapse onto a frontier curve, with performance approaching random at roughly $3D_{\text{train}}$. Thus, deeper training extends the solvable range approximately linearly, but does not eliminate the horizon limit.

5 Conclusion

We introduced SCALELOGIC, a synthetic logical reasoning environment with independent control over reasoning depth and logical expressiveness, and used it to study how RL post-training scales with task complexity. Training compute T follows a power law in proof depth D ($T \propto D^\gamma$, $R^2 > 0.99$) across settings, with the scaling exponent γ increasing monotonically from 1.04 to 2.60 as the logic grows richer. Crucially, expressiveness governs not only training dynamics but also downstream transfer: more expressive settings yield larger and more compute-efficient gains on real-world reasoning tasks, indicating that downstream transfer is shaped by *what* a model is trained on, not merely by *how much* it is trained.

Several directions remain open for future work. First, while our study provides a controlled empirical characterization of RL scaling in long-horizon reasoning, it remains important to test whether these scaling trends persist for substantially larger models and broader RL training regimes. Second, our expressiveness hierarchy covers several core logical operators, but richer fragments—such as equality, higher-order reasoning, non-monotonic reasoning, and more realistic multi-entity relational structures—could reveal new scaling regimes beyond those observed here. Finally, the observed power-law exponents call for a more formal theoretical explanation: understanding why different logical operators change the scaling exponent may provide a principled account of how structural properties of training data shape RL efficiency.

References

- Jiangjie Chen, Qianyu He, Siyu Yuan, Aili Chen, Zhicheng Cai, Weinan Dai, Hongli Yu, Qiying Yu, Xuefeng Li, Jiaze Chen, Hao Zhou, and Mingxuan Wang. Enigmata: Scaling logical reasoning in large language models with synthetic verifiable puzzles. *arXiv preprint arXiv:2505.19914*, 2025.
- Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340. Springer, 2008. doi: 10.1007/978-3-540-78800-3_24.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025a.
- Xiaojun Guo, Ang Li, Yifei Wang, Stefanie Jegelka, and Yisen Wang. G1: Teaching llms to reason on graphs with reinforcement learning. *arXiv preprint arXiv:2505.18499*, 2025b.
- Andre He, Nathaniel Weir, Kaj Bostrom, Allen Nie, Darion Cassel, Sam Bayless, and Huzefa Rangwala. Resyn: Autonomously scaling synthetic environments for reasoning models. *arXiv preprint arXiv:2602.20117*, 2026.
- Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems. *arXiv preprint arXiv:2402.14008*, 2024.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- Tom Henighan, Jared Kaplan, Mor Katz, Mark Chen, Christopher Hesse, Jacob Jackson, Heewoo Jun, Tom B. Brown, Prafulla Dhariwal, Scott Gray, Chris Hallacy, Benjamin Mann, Alec Radford, Aditya Ramesh, Nick Ryder, Daniel M. Ziegler, John Schulman, Dario Amodei, and Sam McCandlish. Scaling laws for autoregressive generative modeling. *arXiv preprint arXiv:2010.14701*, 2020.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. R-zero: Self-evolving reasoning llm from zero data. *arXiv preprint arXiv:2508.05004*, 2025.
- Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.
- Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Devvrit Khatri, Lovish Madaan, Rishabh Tiwari, Rachit Bansal, Sai Surya Duvvuri, Manzil Zaheer, Inderjit S. Dhillon, David Brandfonbrener, and Rishabh Agarwal. The art of scaling reinforcement learning compute for llms. *arXiv preprint arXiv:2510.13786*, 2025.
- Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models. *arXiv preprint arXiv:2206.14858*, 2022.

- Bill Yuchen Lin, Ronan Le Bras, Kyle Richardson, Ashish Sabharwal, Radha Poovendran, Peter Clark, and Yejin Choi. Zebralogic: On the scaling limits of llms for logical reasoning. In *International Conference on Machine Learning*, 2025.
- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Huanyu Liu, Ge Li, Jia Li, Hao Zhu, Kechi Zhang, and Yihong Dong. Saturn: Sat-based reinforcement learning to unleash llms reasoning. *arXiv preprint arXiv:2505.16368*, 2025a.
- Junteng Liu, Yuanxiang Fan, Zhuo Jiang, Han Ding, Yongyi Hu, Chi Zhang, Yiqi Shi, Shitong Weng, Aili Chen, Shiqi Chen, Yunan Huang, Mozhi Zhang, Pengyu Zhao, Junjie Yan, and Junxian He. Synlogic: Synthesizing verifiable reasoning data at scale for learning logical reasoning and beyond. *arXiv preprint arXiv:2505.19641*, 2025b.
- Yi Lu, Jianing Wang, Linsen Guo, Wei He, Hongyin Tang, Tao Gui, Xuanjing Huang, Xuezhi Cao, Wei Wang, and Xunliang Cai. R-horizon: How far can your large reasoning model really go in breadth and depth? *arXiv preprint arXiv:2510.08189*, 2025.
- Trung Quoc Luong, Xinbo Zhang, Zhanming Jie, Peng Sun, Xiaoran Jin, and Hang Li. Reft: Reasoning with reinforced fine-tuning. *arXiv preprint arXiv:2401.08967*, 2024.
- Sumeet Ramesh Motwani, Alesia Ivanova, Ziyang Cai, Philip Torr, Riashat Islam, Shital Shah, Christian Schroeder de Witt, and Charles London. h1: Bootstrapping llms to reason over longer horizons via reinforcement learning. *arXiv preprint arXiv:2510.07312*, 2025.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.
- Andreas Opedal, Yanick Zengaffinen, Haruki Shirakami, Clemente Pasti, Mrinmaya Sachan, Abulhair Saparov, Ryan Cotterell, and Bernhard Schölkopf. Are language models efficient reasoners? a perspective from logic programming. In *Advances in Neural Information Processing Systems*, 2025.
- Revanth Rameshkumar, Jimson Huang, Yunxin Sun, Fei Xia, and Abulhair Saparov. Reasoning models reason well, until they don’t. *arXiv preprint arXiv:2510.22371*, 2025.
- Mohammad Ramezanali, Mo Vazifeh, and Paolo Santi. seqbench: A tunable benchmark to quantify sequential reasoning limits of llms, 2025. URL <https://arxiv.org/abs/2509.16866>.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*, 2023.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv:2409.19256*, 2024.
- Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.
- Zafir Stojanovski, Oliver Stanley, Joe Sharratt, Richard Jones, Abdulhakeem Adefioye, Jean Kaddour, and Andreas Köpf. Reasoning gym: Reasoning environments for reinforcement learning with verifiable rewards. *arXiv preprint arXiv:2505.24760*, 2025.
- Zelin Tan, Hejia Geng, Xiaohang Yu, Mulei Zhang, Guancheng Wan, Yifan Zhou, Qiang He, Xiangyuan Xue, Heng Zhou, Yutao Fan, et al. Scaling behaviors of llm reinforcement learning post-training: An empirical study in mathematical reasoning. *arXiv preprint arXiv:2509.25300*, 2025.
- Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, et al. Kimi k2. 5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.
- Qwen Team. Qwen3.5: Accelerating productivity with native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>.

- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2023.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *Advances in Neural Information Processing Systems*, 37:95266–95290, 2024.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022.
- Shuang Wu, Liwen Zhu, Tao Yang, Shiwei Xu, Qiang Fu, Yang Wei, and Haobo Fu. Enhance reasoning for large language models in the game werewolf. *arXiv preprint arXiv:2402.02330*, 2024.
- Tian Xie, Zitian Gao, Qingnan Ren, Haoming Luo, Yuqian Hong, Bryan Dai, Joey Zhou, Kai Qiu, Zhirong Wu, and Chong Luo. Logic-rl: Unleashing llm reasoning with rule-based reinforcement learning. *arXiv preprint arXiv:2502.14768*, 2025.
- An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *arXiv preprint arXiv:2305.10601*, 2023.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.
- Yang Zhou, Hongyi Liu, Zhuoming Chen, Yuandong Tian, and Beidi Chen. Gsm-infinite: How do your llms behave over infinitely increasing context length and reasoning complexity? In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

Appendix

A	Limitations	14
B	Broader Impact	14
C	Task Generation Procedure	14
C.1	Backward Construction of Proof Trees	14
C.2	Multi-Choice Instance Assembly	15
C.3	Natural Language Conversion	17
D	Detailed Description of Logical Expressiveness	17
D.1	Implication-only	17
D.2	+ Conjunction	17
D.3	+ Negation	17
D.4	+ Disjunction	18
D.5	+ Quantification	18
E	Validity of the Construction	18
E.1	Shortcut Controls	18
E.2	Logical Soundness via Z3 Verification	19
F	Full Implementation Details	19
F.1	Experimental Details	19
F.2	Curriculum Implementation	20
G	Prompt Templates	20
H	Scaling Robustness	20
H.1	Power-Law vs. Exponential Fit Comparison	21
H.2	Sensitivity to the Accuracy Threshold	21
H.3	Different Metrics of Training Compute	22
H.4	Variability across Random Seeds	23
H.5	Multi-Entity Reasoning as a Potential Confounder	24
I	Additional Experimental Results	24
I.1	Candidate Count Scaling	24
I.2	Detailed Downstream Results	25
I.3	Curriculum at the Most Expressive Setting	25
I.4	Data Distribution Shapes Long-CoT Emergence	26
I.5	Cross-Scale Replication on Qwen3-8B	26
J	Full Training Dynamics of the Main Experiments	27
K	Evaluation of Frontier LLMs on SCALELOGIC	28
L	Qualitative Examples	30

A Limitations

All experiments are conducted on Qwen3-4B, with a subset replicated on Qwen3-8B (Appendix I.5) to assess cross-scale consistency. However, it remains unclear whether the observed scaling laws generalize to substantially larger models or different architectures. The power-law relationship $T = a \cdot D^\gamma$ is fitted using limited depth values for each expressiveness level. Due to computational constraints, we cannot further extend the depth range. Although the fits achieve $R^2 > 0.99$ and consistently outperform exponential alternatives (Table 4), this limited range means that the power-law characterization should be interpreted as a high-fidelity empirical description of the observed regime, rather than a robust asymptotic law. In addition, the main scaling curves are fit from single-seed runs; our multi-seed analysis at + *Conjunction* shows similar fitted exponents (Appendix H.4), though broader multi-seed validation is still needed. Both the training-distribution comparison and the cross-algorithm analysis are conducted at selected expressiveness levels; whether these conclusions generalize across all expressiveness levels is not verified. Finally, our analysis is empirical. While we observe a clean power-law dependence and a monotone relationship between logical expressiveness and the scaling exponent γ , we provide no theoretical derivation of these relationships. Establishing the theoretical foundation is left to future work.

B Broader Impact

This work studies RL scaling behavior in a controlled synthetic setting using abstract logical literals with randomly sampled entities and predicates; it does not involve real-world sensitive data or content that could be directly misused. A better understanding of how training complexity and data expressiveness govern RL scaling may help the community design more compute-efficient post-training pipelines. We acknowledge that general improvements in reasoning ability could in principle serve both beneficial and harmful downstream uses, but this risk is shared broadly across reasoning research and is not specific to our contribution.

C Task Generation Procedure

This appendix provides the full generation procedure for SCALELOGIC. We decompose the pipeline into three stages: backward construction of candidate proof trees, assembly of a multiple-choice instance with exactly one provable candidate, and conversion from symbolic formulas to natural language prompts. Throughout the pipeline, the expressiveness flags $\Phi \subseteq \{\wedge, \neg, \vee, \forall\}$ determine which logical operators are available, corresponding to the settings described in Section 3.2. The soundness of the resulting construction is audited in Appendix E.

C.1 Backward Construction of Proof Trees

Algorithm 1 constructs a single candidate proof tree. The generator initializes the work queue as $Q = [(0, \textit{goal})]$, where *goal* is a randomly sampled root literal at depth 0. During construction, Q stores the current open leaves of the partially generated proof tree, each paired with its depth. In practice, Q is processed in a depth-first order: POP always selects a leaf literal at the largest current depth. Once a popped literal has depth $d \geq D$, it is added to the axiom set as a given literal and is no longer expanded. This depth-first expansion guarantees that the proof tree is grown until it reaches the target proof depth D .

For each popped leaf literal u with $d < D$, the generator assigns a rule whose conclusion contains u . The expressiveness flags Φ restrict the admissible rule forms: when conjunction or disjunction is enabled, we allow rules with multiple premises or multiple conclusions, respectively, with the arity set to 2 in our experiments. All newly introduced literals in the premise set P and conclusion set C , except for the already fixed literal u , are assigned fresh predicates to avoid ambiguity, and are treated as new leaf literals added to Q . For details on how disjunctive conclusions are resolved to preserve a unique proof path, see Appendix D. When universal quantification is enabled, the generator may reuse an existing universal rule template from the template set $\mathcal{T}$ with probability $p_{\text{reuse}} = \frac{|\mathcal{T}|}{|\mathcal{T}|+1}$. It instantiates the template with an entity for which the template has not yet been used, and binds one of the instantiated conclusions to the current leaf literal u . All remaining literals introduced by the instantiated rule are then handled in the same way as in ordinary rule expansion. In our experiments,

Algorithm 1: Backward Construction of a Proof Tree

Input: target depth D , expressiveness flags $\Phi \subseteq \{\wedge, \neg, \vee, \forall\}$
Output: axiom-goal pair $(S, goal)$

```
goal ← sample a fresh literal ;           // root conclusion of the proof tree
Q ← [(0, goal)] ;                         // initial depth and root literal
S ← [] ;
T ← [] ;                                  // reusable quantified-rule templates
while Q ≠ ∅ do
  (d, u) ← POP(Q) ;                       // pop max-depth leaf
  if d ≥ D then
    S ← S ∪ {u} ;                         // add a literal axiom
    continue ;
  scope ← CHOOSEENTITYSCOPE(∀ ∈ Φ) ;      // entity-specific or universal
  if scope = ∀ and T ≠ ∅ and BERNOULLI(preuse) then
    r ← instantiate a template from T with a fresh entity ;
    Bind one conclusion of r to u, update S ; enqueue other literals of r into Q ;
    Remove the template from T if it has been instantiated for all entities ;
    continue ;
  (np, nc) ← SAMPLEARITY(Φ) ;           // np ≥ 2 needs ∧ ∈ Φ, nc ≥ 2 needs ∨ ∈ Φ
  P ← BUILDPREMISES(d + 1, np) ;
  C ← BUILDCONCLUSIONS(d, nc, u) ;    // u should be one of the conclusions
  enqueue newly introduced literals from P and C into Q ;
  S ← S ∪ {P → C} ;                     // adds a rule axiom
  if scope = ∀ then
    T ← T ∪ {P → C} ;
if ¬ ∈ Φ then
  foreach literal u in S do
    if BERNOULLI(p¬) then
      negate all occurrences of u in S ; // random polarity flip for ¬
return (S, goal) ;
```

each instance contains at most two entities, so a reused universal template is removed from $\mathcal{T}$ once it has been instantiated for all available entities. After the proof tree is built, if negation is enabled, each literal is independently polarity-flipped with probability $p_{\neg} = 0.5$, and the flip is applied consistently to all of its occurrences in the axiom set. This final polarity randomization changes the surface polarity distribution while preserving the derivability of the constructed proof.

C.2 Multi-Choice Instance Assembly

Algorithm 2 assembles a B -choice problem from B independently generated axiom-goal pairs $\{(S_i, goal_i)\}_{i=1}^B$. Without loss of generality, the first proof tree is kept intact, so $goal_1$ remains provable and serves as the unique correct candidate (the candidate order is randomized later in Appendix C.3). For each remaining proof tree $i = 2, \dots, B$, the generator selects an axiom a^* uniformly at random from S_i and corrupts it. If negation is not enabled, the selected axiom is removed. If negation is enabled, the axiom is either removed with probability $p_{\text{remove}} = 0.5$ or corrupted by uniformly sampling one literal in the axiom and flipping its polarity. Because Algorithm 1 constructs each candidate with a unique proof path, corrupting one selected axiom breaks the only derivation of $goal_i$, making it unprovable. The corrupted axiom set is then merged into the global axiom set S , and $goal_i$ is added to the candidate list.

After merging all B proof trees, the generator optionally adds distractor rules to increase local ambiguity. In our implementation, the total number of rule applications is capped at BD . We sample a distractor count $m \sim \text{Uniform}\{0, \dots, M\}$ with $M = 5$, and then add at most m distractor rules without exceeding the BD edge cap. Each distractor follows the rule forms permitted by Φ , but is constructed so that at least one side of the rule, either the premise set P_d , the conclusion set C_d , or

Algorithm 2: Multiple-Choice Instance Assembly

Input: B axiom-goal pairs $\{(\mathcal{S}_i, goal_i)\}_{i=1}^B$, target depth D , distractor budget M , expressiveness flags Φ
Output: combined axiom set $\mathcal{S}$, candidate conclusions $\mathcal{C}$

```
 $\mathcal{S} \leftarrow \mathcal{S}_1;$   
 $\mathcal{C} \leftarrow \{goal_1\};$  // goal 1 is fixed as provable  
for  $i = 2$  to  $B$  do  
   $a^* \leftarrow$  a randomly selected axiom from  $\mathcal{S}_i$ ;  
  if  $\neg \notin \Phi$  or  $\text{BERNOULLI}(p_{\text{remove}})$  then  
    remove  $a^*$  from  $\mathcal{S}_i$ ;  
  else  
    flip the polarity of one literal in  $a^*$ ; // available only when  $\neg \in \Phi$   
   $\mathcal{S} \leftarrow \mathcal{S} \cup \mathcal{S}_i$ ;  
   $\mathcal{C} \leftarrow \mathcal{C} \cup \{goal_i\}$ ;  
 $m \leftarrow \text{UNIFORM}\{0, \dots, M\}$ ;  
 $k \leftarrow 0$ ;  
while  $k < m$  and  $\text{rules}(\mathcal{S}) < BD$  do  
  randomly choose to introduce fresh distractor literals for  $P_d, C_d$ , or both;  
   $\mathcal{S} \leftarrow \mathcal{S} \cup \{P_d \rightarrow C_d\}$ ;  $k \leftarrow k + 1$ ; // add distractor rules  
return  $(\mathcal{S}, \mathcal{C})$ ;
```

Algorithm 3: Natural Language Conversion

Input: axiom set $\mathcal{S}$, candidate conclusions $\mathcal{C}$
Output: prompt x , answer y

```
 $\phi_e \leftarrow$  random bijection from entity IDs to names; // e.g.,  $e_0 \mapsto \text{"Alice"}$   
 $\phi_p \leftarrow$  random bijection from literal IDs to predicates; // e.g.,  $u_3 \mapsto \text{"furry"}$   
 $F \leftarrow []$ ; // natural-language facts  
foreach  $s \in \text{SHUFFLE}(\mathcal{S})$  do  
  if  $s$  is a given literal then  
     $F.\text{APPEND}(\text{TOTEXT}(s, \phi_e, \phi_p))$ ; // e.g., "Alice is furry."  
  else  
     $f \leftarrow \text{RULETOTEXT}(s, \phi_e, \phi_p)$ ;  
    if  $s$  involves a universal entity then  
       $f \leftarrow \text{QUANTIFY}(f)$ ; // e.g., "If anyone is X, then they are Y."  
     $F.\text{APPEND}(f)$ ;  
 $C_{\text{nl}} \leftarrow []$ ; // natural-language conclusions  
foreach  $c \in \text{SHUFFLE}(\mathcal{C})$  do  
   $C_{\text{nl}}.\text{APPEND}(\text{TOTEXT}(c, \phi_e, \phi_p))$ ;  
 $y \leftarrow$  the unique provable candidate in  $C_{\text{nl}}$ ;  
 $x \leftarrow \text{FORMATPROMPT}(F, C_{\text{nl}})$ ;  
return  $(x, y)$ ;
```

both, contains only fresh predicates not used elsewhere in the instance. If only one side is fresh, the remaining side is sampled from the literals already present in the current instance. For example, a distractor might state “If Alice is a cat, then Alice is small” ($\text{cat}(\text{Alice}) \rightarrow \text{small}(\text{Alice})$) where “small” is a fresh predicate absent from all proof trees. Consequently, each distractor can interact with the existing derivation graph on at most one side: it may be triggered by existing literals but only derive fresh literals, or it may require fresh literals and therefore cannot be triggered by the existing axioms. This increases local ambiguity without changing the unique provable answer.

C.3 Natural Language Conversion

Algorithm 3 converts the symbolic instance into a natural-language multiple-choice prompt. For each instance, we first sample a bijection from internal entity IDs to a fixed pool of 26 names, shown below.

Name pool. Alice, Bob, Carol, David, Emma, Frank, Grace, Henry, Irene, Jack, Kate, Leo, Mona, Nick, Olivia, Paul, Quincy, Rachel, Sam, Tina, Ulysses, Victoria, Wendy, Xavier, Yvonne, and Zach.

We also sample a separate bijection from predicate IDs to random 5-letter predicate strings. These mappings are resampled independently for every instance, preventing the model from relying on fixed entity identities or real-world predicate semantics.

Each symbolic axiom is then rendered with predefined templates: literal axioms become factual statements (e.g., Alice is abcde), grounded rules become if-then statements (e.g., If Alice is abcde, Alice is bcdef), and universal rules are expressed with quantified templates such as “If anyone is abcde, then they are bcdef”. The rendered facts are shuffled before being placed in the prompt, and the candidate conclusions are also shuffled to remove ordering cues. The final answer y is the natural-language rendering of the unique provable candidate, while the prompt x consists of the rendered facts with the shuffled candidate list using the templates in Appendix G.

D Detailed Description of Logical Expressiveness

This section provides the full description of the five expressiveness levels summarized in the main text (Section 3.2). Table 2 summarizes these levels and their features.

Table 2: Mapping between the five levels of logical expressiveness that we consider and their features.

Setting	$\rightarrow$	$\wedge$	$\neg$	$\vee$	$\forall$
Implication-only	✓	–	–	–	–
+ Conjunction	✓	✓	–	–	–
+ Negation	✓	✓	✓	–	–
+ Disjunction	✓	✓	✓	✓	–
+ Quantification	✓	✓	✓	✓	✓

D.1 Implication-only

In the implication-only setting, each axiom is either a grounded literal or a simple implication rule. Starting from the literal axioms, reasoning proceeds by repeatedly applying rules whose antecedents are satisfied, thereby deriving new literals. A statement is considered valid if it can be derived from the axioms through a sequence of such rule applications. Performing deductive reasoning under this logic is equivalent to path-finding in simple directed graphs.

D.2 + Conjunction

In this setting, a rule may take the form of a conjunction of grounded literals entailing a single conclusion. Accordingly, at inference time, the model must jointly track and verify multiple supporting literals at each reasoning step. Compared to implication-only reasoning, this shifts the task from single-premise inference to coordinating multiple supporting literals before each rule application. Performing deductive reasoning under this logic is equivalent to path-finding in simple directed hypergraphs where each hyperedge may have more than one source node.

D.3 + Negation

In this setting, rules may condition on the absence of a property or derive such an absence as a consequence of other literals. Compared to the conjunction setting, negation further requires the model to maintain polarity information throughout the proof, introducing a qualitatively dif-

ferent form of reasoning.⁵ As a byproduct, negation provides a natural mechanism for constructing non-target candidate conclusions: flipping the polarity of an intermediate literal within a proof-tree axiom breaks the derivation and renders the root conclusion unprovable. For example, $\text{mammal}(\text{Alice}) \rightarrow \neg \text{bird}(\text{Alice})$ can be corrupted into either $\neg \text{mammal}(\text{Alice}) \rightarrow \neg \text{bird}(\text{Alice})$ or $\text{mammal}(\text{Alice}) \rightarrow \text{bird}(\text{Alice})$, depending on whether the sampled literal is the premise or the conclusion (cf. Section 3.1; algorithm is provided in Appendix C.2).

D.4 + Disjunction

In this setting, a rule may take the form of the antecedents entailing a disjunction of grounded literals. Applying such a rule yields a set of candidate conclusions rather than a single deterministic outcome. A consequence of this is that the simple proof tree analogy described in Section 3.1 must be extended: With disjunction, each proof step may have more than one literal as the conclusion. As such, each edge in the proof tree must be generalized as a hyperedge, which can have more than one parent node and more than one child node. This creates an ambiguity that is absent in ordinary proof trees: a disjunctive rule establishes a disjunction, but not any individual disjunct by itself. For example, suppose the current root literal is $\text{mammal}(\text{Alice})$, and the backward construction introduces the rule $\text{cat}(\text{Alice}) \rightarrow \text{mammal}(\text{Alice})$, making $\text{cat}(\text{Alice})$ a literal that must be supported. If we then try to support this literal using $\text{pet}(\text{Alice}) \rightarrow \text{cat}(\text{Alice}) \vee \text{dog}(\text{Alice})$ together with $\text{pet}(\text{Alice})$, we can derive $\text{cat}(\text{Alice}) \vee \text{dog}(\text{Alice})$, but not $\text{cat}(\text{Alice})$ itself. Therefore, the original root literal $\text{mammal}(\text{Alice})$ is not guaranteed to be derivable unless the alternative disjunct $\text{dog}(\text{Alice})$ is resolved.

To keep the proof well-defined during our “backwards” proof generation procedure, we resolve each alternative disjunct in one of two standard ways: (i) it is ruled out by additional axioms that contradict it (e.g., a separate axiom states $\neg \text{dog}(\text{Alice})$), or (ii) it eventually leads to a common downstream conclusion shared with the chosen disjunct (e.g., add a rule $\text{dog}(\text{Alice}) \rightarrow \text{mammal}(\text{Alice})$ so that both $\text{cat}(\text{Alice})$ and $\text{dog}(\text{Alice})$ lead to $\text{mammal}(\text{Alice})$). At inference time, the model must therefore examine which disjuncts are eliminated and which converge on a shared conclusion, shifting the task from committing to a single outcome at each step to reasoning over a set of possible conclusions.

D.5 + Quantification

In this setting, universal quantification enables defining rules that apply to *any* entity rather than to a specific one. We also consider the number of entities in the problem as an additional axis of complexity. In many of our experiments, we consider the simplest case where all generated literals apply to a single entity (e.g., Alice). We also consider a multi-entity setting, where axioms may involve several distinct entities within the same problem (e.g., Alice and Bob); by default, we use two entities. The use of multiple entities is particularly important when rules are universally quantified: without multiple entities, quantified rules reduce to entity-specific ones and offer no additional reasoning challenge. With multiple entities present, we additionally enable *predicate and rule reuse*: the same predicate may apply to different entities, and a single quantified rule may be instantiated multiple times within the same proof, creating compositional overlap that requires the model to disambiguate shared structure rather than treating each occurrence independently. Together, these extensions move the environment from basic propositional inference towards the more expressive regime of first-order reasoning.

E Validity of the Construction

We provide two complementary checks to ensure that our synthetic problems are free of surface-form shortcuts and are logically sound.

E.1 Shortcut Controls

We control surface-form shortcuts through the generation and rendering procedure described in Appendix C. Predicates are sampled as fresh random strings and entity-name mappings are resampled

⁵We note that our proofs do not contain hypothetical derivations, such as proofs by contradiction.

Table 3: Default RL Training Hyperparameters

Parameter	Value
Generate batch size	384 prompts
Training batch size	256 prompts
Mini batch size	64 prompts
Samples per prompt	8
Max response length	8192
Clip ratio	(0.20, 0.28)
Temperature	1.0
Top p	1.0
Optimizer	AdamW
KL coefficient	0
Actor learning rate	1×10^{-6}
LR warmup steps	10
LR scheduler	constant

independently for each instance, preventing stable lexical or world-knowledge cues. For unprovable candidates, the corrupted axiom is selected uniformly from the corresponding proof tree, preventing the position of the corrupted axiom from revealing the answer. When negation is enabled, polarity-flip corruptions are also randomized, preventing polarity statistics from serving as a shortcut. Finally, both the axiom order and candidate order are shuffled before rendering, preventing ordering cues in the prompt or answer options. These controls are designed to ensure that the correct answer is determined by logical derivability rather than by surface statistics.

E.2 Logical Soundness via Z3 Verification

Although the labels produced by our backward-construction procedure are correct by construction, implementation imperfections may still introduce unwanted label noise. To further verify the correctness of the labels, we audit a random subset of generated problems using the Z3 SMT solver [de Moura and Bjørner, 2008]. For each sampled problem, we encode the axiom set as a first-order theory in Z3 and test entailment for every candidate literal. An instance passes the audit only if Z3 confirms that the marked-provable candidate is entailed and all $B - 1$ marked-unprovable candidates are not entailed. We sample $N = 1000$ problems from every configuration used in our experiments, and all sampled instances pass the audit, confirming that the labels used during RL training are logically reliable.

F Full Implementation Details

F.1 Experimental Details

Training. We conduct all RL post-training with the `ver1` library [Sheng et al., 2024] on the non-thinking version of Qwen3-4B and Qwen3-8B [Yang et al., 2025], running on $8 \times B200$ 180G GPUs. Unless otherwise specified, all experiments share the training recipe described in Section 3.3, with hyperparameters summarized in Table 3. Full training dynamics are provided in Appendix J.

Evaluation. For all downstream benchmarks, we follow the official decoding parameters recommended by Qwen [Yang et al., 2025]: temperature $T = 0.7$, top- $p = 0.8$, and top- $k = 20$. The maximum response length is set to 8192 tokens, matching the training budget. We record Avg@8 and Pass@8 for each benchmark and report the mean accuracy across benchmarks (i.e., the mean of per-benchmark Avg@8 scores) as our primary metric in Section 4.3.

For the out-of-distribution generalization experiments in Section 4.6, where evaluation depths can substantially exceed the training depth, we increase the maximum response length to 32,768 tokens. We find this is necessary because solving deeper OOD problems requires substantially longer reasoning than seen during training, and a tighter budget would truncate completions before they

reach an answer, conflating reasoning capability with generation-length limits. All other decoding settings remain unchanged; we also report Avg@8 as the main metric.

F.2 Curriculum Implementation

We implement the curriculum strategy used in Section 4.4 following the threshold-triggered scheme of Stojanovski et al. [2025]. At each training step, tasks are sampled uniformly from proof depths $\{1, \dots, D_{\text{cur}}\}$, where D_{cur} is the current curriculum depth; instances beyond D_{cur} are filtered out. Whenever the rolling training accuracy reaches 70%, we increase D_{cur} by a fixed step size Δ , until it reaches the maximum training depth D_{max} . We use the curriculum hyperparameters in the following table. The initial D_{cur} is chosen as the smallest depth at which the base model is not yet saturated; shallower depths are already solved at or near the threshold and would trigger immediate curriculum advancement. We focus on demonstrating the effectiveness of curriculum-based training; designing more efficient curricula and selecting their hyperparameters more systematically is left to future work.

Setting	D_{cur}	Δ
+ Conjunction	8	4
+ Quantification	4	2

For comparison, the default uniform setting is trained on depths uniformly distributed over $1, \dots, D_{\text{max}}$ throughout, while *difficult-only* training is trained exclusively on depth D_{max} . All three strategies use identical training hyperparameters (Table 3).

G Prompt Templates

We provide the prompt templates used for both training and downstream evaluation, ensuring consistency and reproducibility across tasks.

Prompt Template for Logical Reasoning Task

Suppose we have the following facts:

{List of shuffled axioms and rules}

Here are some candidate statements:

{List of shuffled candidate statements}

Exactly one of the above statements can be logically derived from the given facts. Identify that statement and return it within <answer> </answer> tags. Example: <answer> Alice is abcde </answer>.

Prompt Template for Math Benchmarks

{Problem}

Please reason step by step, and put your final answer within `\boxed{ }`.

Prompt Template for GPQA-Diamond and MMLU-Pro

{Problem}

Please reason step by step, and put your final answer within `\boxed{ }`. Example: `\boxed{A}`.

H Scaling Robustness

We conduct five complementary checks to verify that our main scaling claims are robust to key design and analysis choices.

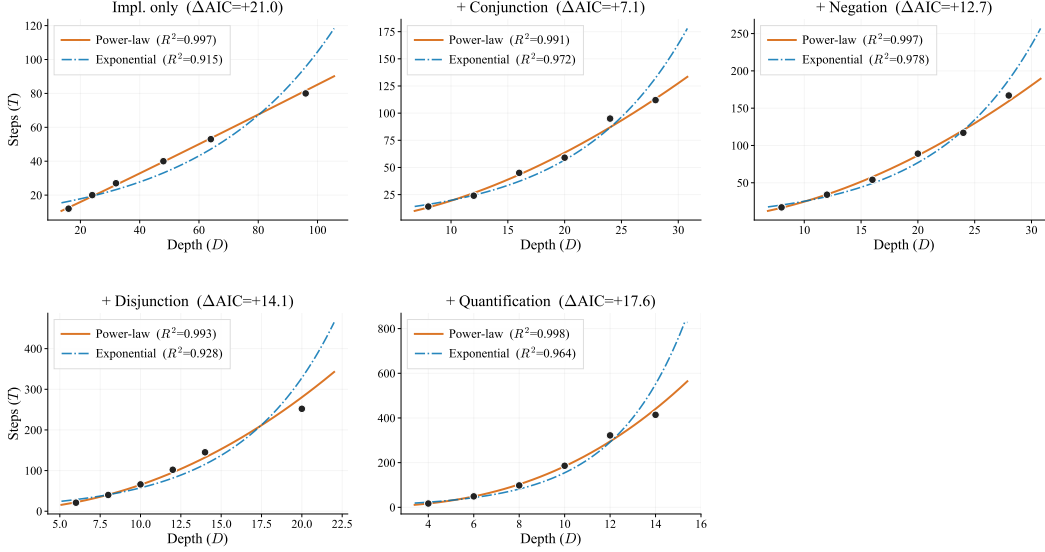


Figure 6: Power-law (orange) vs. exponential (blue) fits for each expressiveness setting. ΔAIC values consistently favor the power-law model.

H.1 Power-Law vs. Exponential Fit Comparison

To validate the power-law scaling relationship reported in Section 4.2, we compare power-law and exponential fits for each expressiveness setting. Both models are fit via ordinary least squares in log space with two free parameters each, enabling direct comparison through the Akaike Information Criterion (AIC).

Table 4: Power-law ($T = a \cdot D^\gamma$) vs. exponential ($T = a \cdot e^{bD}$) fit comparison across expressiveness levels. Both models use two free parameters and are fit via OLS in log space. $\Delta\text{AIC} = \text{AIC}_{\text{exp}} - \text{AIC}_{\text{pow}}$; positive values favor the power-law model.

Setting	Power-law		Exponential		ΔAIC
	$\gamma \pm \text{SE}$	R^2	$b \pm \text{SE}$	R^2	
Implication-only	1.04 ± 0.03	0.997	0.022 ± 0.003	0.916	+21.0
+ Conjunction	1.72 ± 0.08	0.991	0.106 ± 0.009	0.972	+7.1
+ Negation	1.81 ± 0.05	0.997	0.112 ± 0.008	0.978	+12.7
+ Disjunction	2.11 ± 0.09	0.993	0.174 ± 0.024	0.929	+14.1
+ Quantification	2.60 ± 0.06	0.998	0.318 ± 0.031	0.964	+17.6

As shown in Table 4 and Figure 6, the power-law model achieves higher R^2 and lower AIC than the exponential model across all five settings ($\Delta\text{AIC} \geq +7.1$). Figure 6 visualizes both fits against the observed data, showing that the exponential model systematically overshoots at large depths while the power-law fit remains accurate throughout.

H.2 Sensitivity to the Accuracy Threshold

To verify that our scaling claims are not artifacts of the specific 90% accuracy threshold used in the main text, we refit $T \propto D^\gamma$ using a lower threshold of 85% (Figure 7). As shown in Table 5, the fitted exponents shift only modestly across all five settings.

All five settings continue to admit a power-law fit with $R^2 > 0.99$, and γ remains monotonically increasing in expressiveness ($1.07 \rightarrow 2.73$). The relative ordering of the five exponents is preserved exactly. Our main conclusions are therefore not sensitive to the specific accuracy threshold used.

Table 5: Power-law fits at the main-text accuracy threshold ($\mu = 90\%$) versus a relaxed threshold ($\mu = 85\%$). All five expressiveness settings preserve $R^2 > 0.99$ under both thresholds, and the relative ordering of the fitted exponents is preserved exactly.

Setting	$\mu = 90\%$		$\mu = 85\%$	
	$\gamma \pm \text{SE}$	R^2	$\gamma \pm \text{SE}$	R^2
Implication-only	1.04 ± 0.03	0.997	1.07 ± 0.04	0.994
+ Conjunction	1.72 ± 0.08	0.991	1.80 ± 0.08	0.993
+ Negation	1.81 ± 0.05	0.997	1.88 ± 0.04	0.998
+ Disjunction	2.11 ± 0.09	0.993	2.19 ± 0.11	0.991
+ Quantification	2.60 ± 0.06	0.998	2.73 ± 0.06	0.998

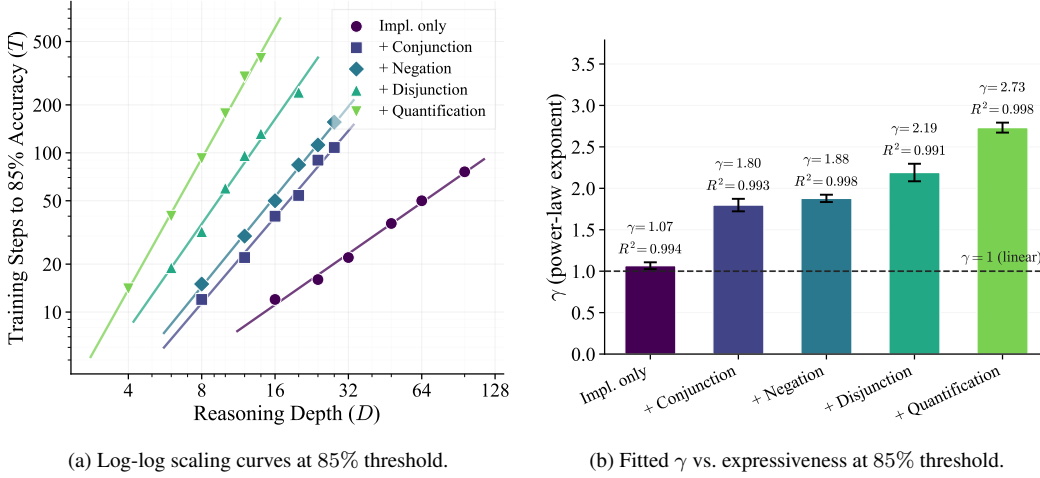


Figure 7: Power-law scaling under 85% accuracy threshold (cf. main-text Figure 2 for 90%). The qualitative pattern is unchanged: all five settings follow a power law ($R^2 > 0.99$), and γ increases monotonically with expressiveness.

H.3 Different Metrics of Training Compute

The main text uses RL training steps as the measure of training effort T . To check that our scaling claims do not depend on this choice, we refit $T \propto D^\gamma$ under four alternative compute measures: total generated tokens across all rollouts including rejected samples $T_{\text{gen-tok}}$; prompt and response tokens in the kept batch $T_{\text{upd-tok}}$; training FLOPs by the Kaplan rule [Kaplan et al., 2020], $T_{\text{FLOPs}} = 2N T_{\text{gen-tok}} + 6N T_{\text{upd-tok}}$ with N the model’s parameter count; and wall-clock GPU-hours on 8×B200.

Table 6: Power-law fits $T(D) = a \cdot D^\gamma$ for four compute-effort measures. $T_{\text{gen-tok}}$ counts generated tokens over all rollouts (including rejected samples); $T_{\text{upd-tok}}$ counts the prompt and response tokens of the kept batch only; $T_{\text{FLOPs}} = 2N T_{\text{gen-tok}} + 6N T_{\text{upd-tok}}$ follows the Kaplan rule; GPU-hrs is wall-clock $\times 8$ B200 GPUs.

Setting	$T_{\text{gen-tok}}$		$T_{\text{upd-tok}}$		T_{FLOPs}		GPU-hrs	
	$\gamma \pm \text{SE}$	R^2	$\gamma \pm \text{SE}$	R^2	$\gamma \pm \text{SE}$	R^2	$\gamma \pm \text{SE}$	R^2
Implication-only	1.15 ± 0.08	0.980	1.54 ± 0.04	0.998	1.42 ± 0.05	0.995	1.38 ± 0.05	0.994
+ Conjunction	2.18 ± 0.15	0.982	2.39 ± 0.11	0.991	2.33 ± 0.12	0.989	2.40 ± 0.13	0.989
+ Negation	2.16 ± 0.08	0.995	2.33 ± 0.04	0.999	2.28 ± 0.05	0.998	2.33 ± 0.06	0.998
+ Disjunction	2.42 ± 0.15	0.982	2.72 ± 0.11	0.992	2.62 ± 0.12	0.990	2.61 ± 0.15	0.984
+ Quantification	2.97 ± 0.17	0.987	3.19 ± 0.13	0.994	3.12 ± 0.14	0.992	3.09 ± 0.16	0.990

Table 6 and Figure 8 report the fits. The power-law form holds in every cell ($R^2 > 0.98$), and the overall ranking, *Implication-only* < *+ Conjunction*, *+ Negation* < *+ Disjunction* < *+ Quantification*, is preserved across all four measures. Exponents shift upward relative to step counts (e.g., *+ Quan-*

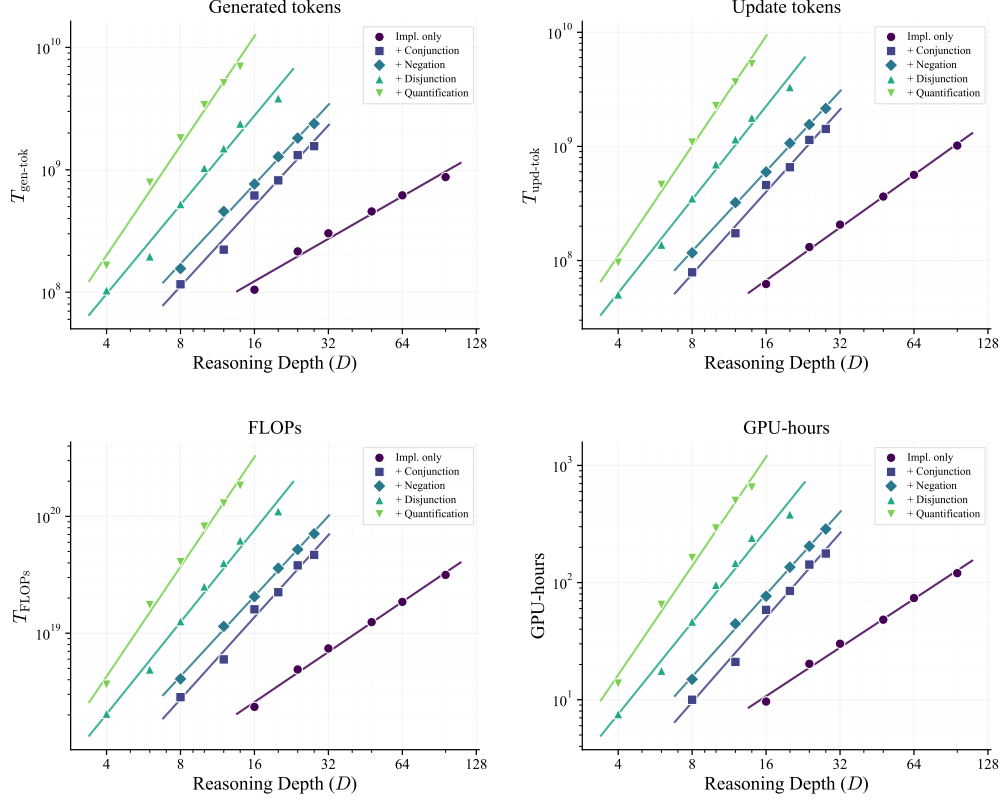


Figure 8: Power-law fits $T(D) = a \cdot D^\gamma$ under four alternative compute measures, complementing Table 6. **Top-left:** generated tokens $T_{\text{gen-tok}}$. **Top-right:** kept-batch prompt and response tokens $T_{\text{upd-tok}}$. **Bottom-left:** training FLOPs $T_{\text{FLOPs}} = 2N T_{\text{gen-tok}} + 6N T_{\text{upd-tok}}$. **Bottom-right:** wall-clock GPU-hours on $8 \times \text{B200}$. All four measures preserve the power-law form ($R^2 > 0.98$).

tification shifts from $\gamma = 2.60$ to $\gamma = 3.12$ under FLOPs), reflecting that token- and FLOP-based measures aggregate per-instance length on top of the number of gradient updates: prompt length grows with depth and expressiveness, and response length grows further during training. + *Conjunction* and + *Negation* remain within each other’s standard-error ranges across all metrics, consistent with the main-text finding that the two impose comparable training burdens.

We retain training steps as our primary measure because they isolate the convergence rate from confounds in per-instance length. Token- and FLOP-based measures aggregate two such confounds: prompts grow structurally with reasoning depth and expressiveness (more axioms, longer rules, quantified templates), and within-training response length grows with both training progress and task difficulty (Figure 13). Conflating these with training effort would mix structural and dynamical properties of the training environment into the scaling measurement.

H.4 Variability across Random Seeds

The main scaling experiment in Section 4.2 uses a single random seed per configuration due to computational constraints. To verify that this does not bias the fitted exponent, we leverage the multi-seed runs from the training-distribution and cross-algorithm experiments (Sections 4.4 and 4.5), which use three seeds per depth on the + *Conjunction* setting. Pooling all three seeds and refitting yields $\gamma = 1.70$ ($R^2 = 0.995$), within 0.02 of the single-seed value reported in the main text ($\gamma = 1.72$, $R^2 = 0.991$). The marginally higher R^2 under pooling likely reflects per-depth variance being averaged out before fitting. This confirms that our single-seed scaling estimates are reliable.

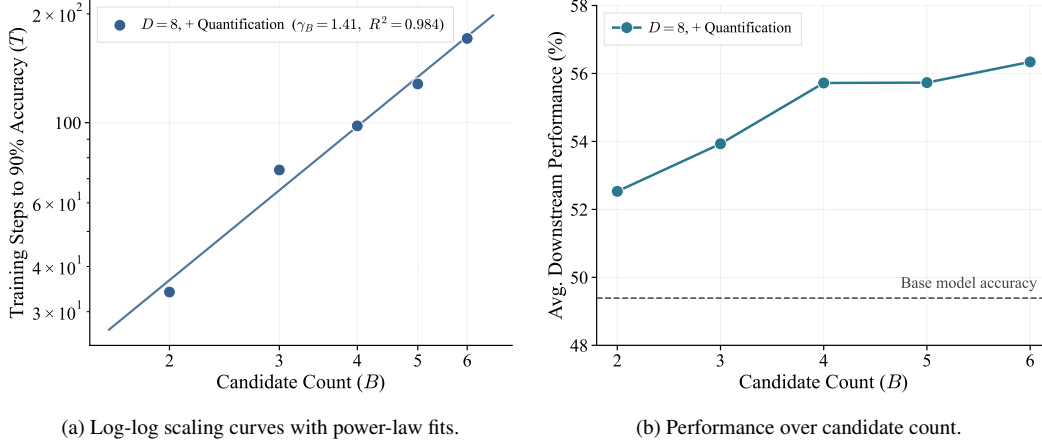


Figure 9: Effect of candidate count B at fixed depth $D = 8$ under the + *Quantification* setting. **(a)** Training steps to 90% accuracy follow a power law in B ($\gamma_B = 1.41$, $R^2 = 0.984$, $\Delta\text{AIC} = +7.0$ vs. exponential). **(b)** Average downstream performance across the eight reasoning benchmarks of Section 4.3, plotted against candidate count B . Gains saturate quickly: performance rises from 52.5% at $B = 2$ to 55.7% at $B = 4$ (+3.2 pp), but only an additional +0.6 pp from $B = 4$ to $B = 6$.

H.5 Multi-Entity Reasoning as a Potential Confounder

The most expressive setting (+ *Quantification*) introduces multiple design elements simultaneously (Appendix D). A natural concern is that the elevated γ at + *Quantification* could be partially driven by the multi-entity dimension rather than by quantification itself.

To isolate the multi-entity contribution, we run an ablation that varies only the number of entities while keeping $\forall$ and reuse disabled (i.e., starting from the + *Disjunction* setting). As shown in Table 7, training steps to 90% accuracy are essentially identical between the single-entity (1 person) and multi-entity (2 persons) variants across $D \in \{6, 8, 10\}$, with absolute differences of at most 2 steps and no systematic trend with depth.

Table 7: Training steps to 90% accuracy under the + *Disjunction* setting with single vs. multiple entities. Differences are within ± 2 steps and show no systematic trend with depth, indicating that the multi-entity dimension by itself does not measurably increase training difficulty.

D	Single entity (1 person)	Multiple entities (2 persons)
6	21	21
8	39	40
10	68	66

We conclude that the multi-entity design by itself contributes negligibly to training difficulty in this depth range, and that the elevated γ at + *Quantification* therefore reflects the additional difficulty imposed by universal quantification and rule reuse rather than by the multi-entity dimension.

I Additional Experimental Results

I.1 Candidate Count Scaling

We examine how the candidate count B (i.e., the number of candidate conclusions per problem) affects both training compute and downstream transfer at fixed depth $D = 8$ under the + *Quantification* setting.

As with depth scaling (Section 4.2), we fit $T = a \cdot B^{\gamma_B}$ via OLS in log-log space. As shown in Figure 9a, training compute follows a power law in B with exponent $\gamma_B = 1.41 \pm 0.10$ and $R^2 = 0.984$. The power-law model again outperforms an exponential fit ($\Delta\text{AIC} = +7.0$). The sub-quadratic exponent indicates that increasing the number of candidate branches is less costly than

Table 8: Detailed downstream results for all benchmarks and training settings. Each entry reports Avg@8 / Pass@8 in percentage. The last column reports the mean accuracy across the eight benchmarks (i.e., the mean of per-benchmark Avg@8 scores).

D	Math						Science		Avg
	Math500	AMC23	AIME24	AIME25	Minerva	Olympiad	GPQA-Diamond	MMLU-Pro	
Base model									
–	83.93 / 93.60	65.00 / 87.50	21.67 / 40.00	20.00 / 40.00	40.21 / 50.74	53.32 / 71.36	42.55 / 76.26	68.45 / 78.60	49.39
Implication-only									
16	84.28 / 94.00	64.69 / 87.50	22.92 / 43.33	22.50 / 43.33	40.40 / 49.63	53.43 / 71.07	42.93 / 76.26	67.85 / 87.20	49.88
24	84.85 / 94.60	68.44 / 90.00	24.58 / 46.67	21.67 / 46.67	40.85 / 50.00	54.32 / 73.15	43.62 / 75.25	69.90 / 88.60	51.03
32	84.95 / 95.60	68.44 / 90.00	24.17 / 50.00	22.50 / 46.67	40.63 / 50.00	54.78 / 71.81	44.32 / 72.73	69.05 / 87.80	51.11
48	84.73 / 94.60	67.19 / 87.50	21.67 / 40.00	20.42 / 43.33	40.81 / 51.10	55.51 / 72.70	43.18 / 77.27	69.05 / 88.40	50.32
64	86.20 / 96.00	70.31 / 97.50	28.75 / 43.33	21.25 / 46.67	41.27 / 50.37	55.14 / 73.00	42.55 / 77.78	70.55 / 89.40	52.00
96	85.53 / 95.20	70.31 / 97.50	23.75 / 53.33	24.17 / 50.00	40.40 / 51.84	55.17 / 73.00	44.32 / 77.78	70.03 / 88.80	51.71
+ Conjunction									
8	84.30 / 95.00	70.63 / 95.00	26.25 / 46.67	22.50 / 50.00	39.75 / 50.74	53.75 / 71.51	42.36 / 75.25	69.10 / 87.80	51.08
12	85.53 / 95.40	69.69 / 95.00	30.00 / 60.00	20.00 / 40.00	40.40 / 49.63	54.67 / 73.74	41.60 / 77.27	69.55 / 88.80	51.43
16	85.60 / 95.00	71.88 / 95.00	24.17 / 43.33	24.17 / 56.67	40.53 / 51.10	55.42 / 73.15	43.06 / 78.79	70.53 / 88.00	51.92
20	85.45 / 95.20	70.94 / 92.50	26.67 / 53.33	21.25 / 46.67	39.66 / 49.63	55.25 / 74.18	44.44 / 79.29	69.20 / 87.80	51.61
24	85.83 / 94.80	72.81 / 97.50	24.17 / 40.00	19.58 / 36.67	40.76 / 50.37	55.84 / 71.81	45.52 / 77.27	69.68 / 87.60	51.77
28	86.15 / 95.60	70.94 / 97.50	27.92 / 53.33	22.08 / 43.33	40.63 / 50.37	55.40 / 74.33	42.42 / 76.26	70.15 / 88.20	51.96
+ Negation									
8	84.98 / 95.20	71.88 / 95.00	22.92 / 50.00	20.42 / 43.33	39.52 / 50.00	54.56 / 73.44	43.06 / 76.77	69.55 / 89.20	50.86
12	85.83 / 96.00	71.56 / 95.00	26.25 / 53.33	20.42 / 43.33	40.53 / 50.74	55.77 / 73.44	44.00 / 76.26	69.25 / 87.40	51.70
16	86.13 / 95.60	72.19 / 90.00	29.17 / 50.00	20.42 / 36.67	40.72 / 52.21	56.64 / 73.89	43.94 / 75.76	70.10 / 89.00	52.41
20	86.85 / 95.40	74.38 / 97.50	30.00 / 53.33	22.92 / 50.00	41.73 / 52.21	56.81 / 73.59	46.72 / 80.30	71.93 / 89.60	53.92
24	87.43 / 96.40	72.81 / 92.50	30.00 / 53.33	22.50 / 50.00	40.72 / 51.47	57.07 / 74.63	44.07 / 75.76	72.63 / 89.60	53.40
28	87.38 / 97.00	73.75 / 92.50	27.08 / 46.67	20.83 / 43.33	40.95 / 51.10	57.62 / 76.41	45.39 / 78.28	72.08 / 87.60	53.13
+ Disjunction									
6	85.48 / 95.40	70.31 / 90.00	24.58 / 46.67	22.92 / 46.67	39.80 / 50.37	54.95 / 73.15	42.30 / 77.27	68.63 / 87.80	51.12
8	86.75 / 96.20	70.00 / 95.00	25.83 / 50.00	25.42 / 53.33	41.41 / 50.00	56.88 / 73.15	46.15 / 78.79	70.53 / 88.60	52.87
10	87.40 / 96.40	71.56 / 95.00	27.50 / 50.00	25.42 / 50.00	40.81 / 51.10	57.60 / 74.33	45.08 / 79.29	72.18 / 88.40	53.44
12	87.20 / 95.60	74.38 / 95.00	30.00 / 50.00	27.50 / 53.33	41.27 / 52.21	57.86 / 73.89	44.07 / 77.78	72.00 / 89.60	54.29
14	87.78 / 96.00	76.56 / 95.00	33.33 / 63.33	25.83 / 50.00	41.27 / 51.84	58.88 / 76.41	45.27 / 80.30	72.85 / 89.60	55.22
20	88.63 / 96.60	76.25 / 95.00	35.00 / 63.33	27.92 / 56.67	41.82 / 51.84	59.20 / 75.82	46.53 / 81.31	74.08 / 89.40	56.18
+ Quantification									
4	85.43 / 95.40	69.38 / 92.50	24.58 / 46.67	23.33 / 50.00	40.17 / 50.37	55.06 / 73.15	41.86 / 77.78	69.47 / 88.20	51.16
6	86.75 / 96.80	70.00 / 90.00	28.33 / 50.00	21.67 / 40.00	40.67 / 51.47	56.82 / 74.78	45.79 / 75.25	71.53 / 88.60	52.69
8	88.15 / 96.40	78.12 / 97.50	35.00 / 56.67	25.00 / 53.33	41.68 / 51.84	58.90 / 75.22	46.34 / 76.77	72.53 / 89.80	55.72
10	88.60 / 97.20	75.94 / 97.50	34.58 / 60.00	25.83 / 53.33	42.33 / 51.84	60.05 / 76.11	45.27 / 76.77	74.73 / 89.80	55.92
12	88.70 / 96.60	80.90 / 97.50	35.00 / 60.00	27.92 / 46.67	42.42 / 51.84	61.13 / 77.45	49.12 / 75.76	74.73 / 89.20	57.49
14	91.48 / 98.00	80.63 / 97.50	43.75 / 66.67	31.25 / 53.33	43.70 / 52.57	64.39 / 79.08	48.17 / 72.22	77.00 / 89.80	60.05

increasing reasoning depth under the same expressiveness setting (where $\gamma = 2.60$), consistent with the intuition that deeper proofs require compositionally harder reasoning while additional branches primarily expand the search space.

Larger B also yields stronger downstream transfer, though with diminishing returns. As shown in Figure 9b, mean downstream accuracy (averaged over the eight benchmarks of Section 4.3) rises from 52.5% at $B = 2$ to 55.7% at $B = 4$ (+3.2 pp), then by only an additional +0.6 pp from $B = 4$ to $B = 6$. This contrasts with the depth axis under the same + *Quantification* setting, where deeper training continues to yield downstream gains throughout our tested range (up to 414 training steps), suggesting that B is a less capacious axis of difficulty than depth. A small number of unprovable candidates is plausibly sufficient to force the model to evaluate each branch, after which additional candidates raise training cost without meaningfully extending the reasoning skills required. We accordingly fix $B = 4$ as the default candidate count in our main experiments.

I.2 Detailed Downstream Results

Table 8 reports the full downstream results for each benchmark and training setting. In Section 4.3, we present the corresponding averaged results across benchmarks for clarity.

I.3 Curriculum at the Most Expressive Setting

The training-distribution comparison in Section 4.4 is conducted under the + *Conjunction* setting. To verify that the benefit of curriculum training is not specific to this setting, we replicate the uniform vs. curriculum comparison at the most expressive + *Quantification* setting.

As shown in Figure 10, curriculum training continues to lower the scaling exponent at + *Quantification*, from $\gamma = 2.60$ to $\gamma = 2.30$, with $R^2 = 0.998$ in both cases. The reduction in exponent ($\Delta\gamma = -0.30$) is comparable to that observed at + *Conjunction* ($\Delta\gamma = -0.37$, from 1.70 to 1.33), and the qualitative effect is preserved: curriculum training yields sample-efficiency gains that compound with proof depth. We conclude that the benefit of curriculum training generalizes across the expressiveness hierarchy.

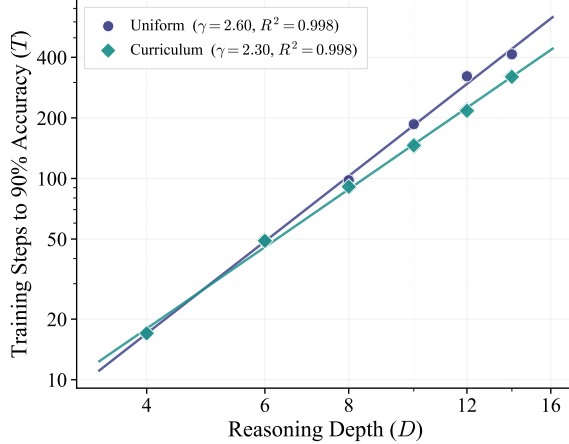


Figure 10: Uniform vs. curriculum training under the most expressive + *Quantification* setting. Curriculum training reduces the scaling exponent from $\gamma = 2.60$ to $\gamma = 2.30$ ($R^2 = 0.998$ in both cases), confirming that the benefit observed at + *Conjunction* (Section 4.4) extends to the most expressive logic in our hierarchy.

I.4 Data Distribution Shapes Long-CoT Emergence

The main text shows that the training distribution substantially affects scaling efficiency in the + *Conjunction* setting. Here, we provide the full training trajectories behind this comparison and analyze how the three data distributions shape models’ behavior. We focus on response length, actor entropy, and validation accuracy, which together reveal when longer reasoning traces emerge and how this transition relates to the observed scaling behavior.

Figure 11 shows that curriculum training substantially eliminates the delay in long-CoT emergence across depths. Under the curriculum regime, runs at different target depths enter the long-CoT regime within a relatively narrow range of training steps; this length expansion is accompanied by a sharper entropy drop and earlier improvement in validation accuracy. By contrast, under the uniform distribution, long-CoT emergence becomes increasingly delayed as the target depth grows, with a particularly large gap between $D = 20$ and $D = 24$; the corresponding entropy decrease and accuracy rise are likewise shifted to much later training steps. The difficult-only regime exhibits a similar but more pronounced pattern, especially at larger depths, where response length remains short and entropy stays elevated for a longer warm-up period before accuracy begins to improve.

Together, these dynamics suggest that the distribution effect in the main text is closely tied to when long-CoT behavior emerges. Curriculum training appears to provide a smoother path into longer reasoning: the policy first learns useful reasoning patterns on shallower instances and can then extend them as deeper instances are introduced. By contrast, uniform and difficult-only training tend to remain longer in a short-response, high-entropy regime before transitioning to long-CoT behavior. This delayed transition is consistent with their larger scaling exponents, whereas the earlier and more synchronized transition under curriculum training helps explain its improved scaling efficiency.

I.5 Cross-Scale Replication on Qwen3-8B

To assess whether the scaling behavior reported in the main text generalizes beyond the 4B scale, we replicate the central scaling experiment of Section 4.2 on Qwen3-8B. Figure 12 shows the log-log scaling curves and the fitted exponents.

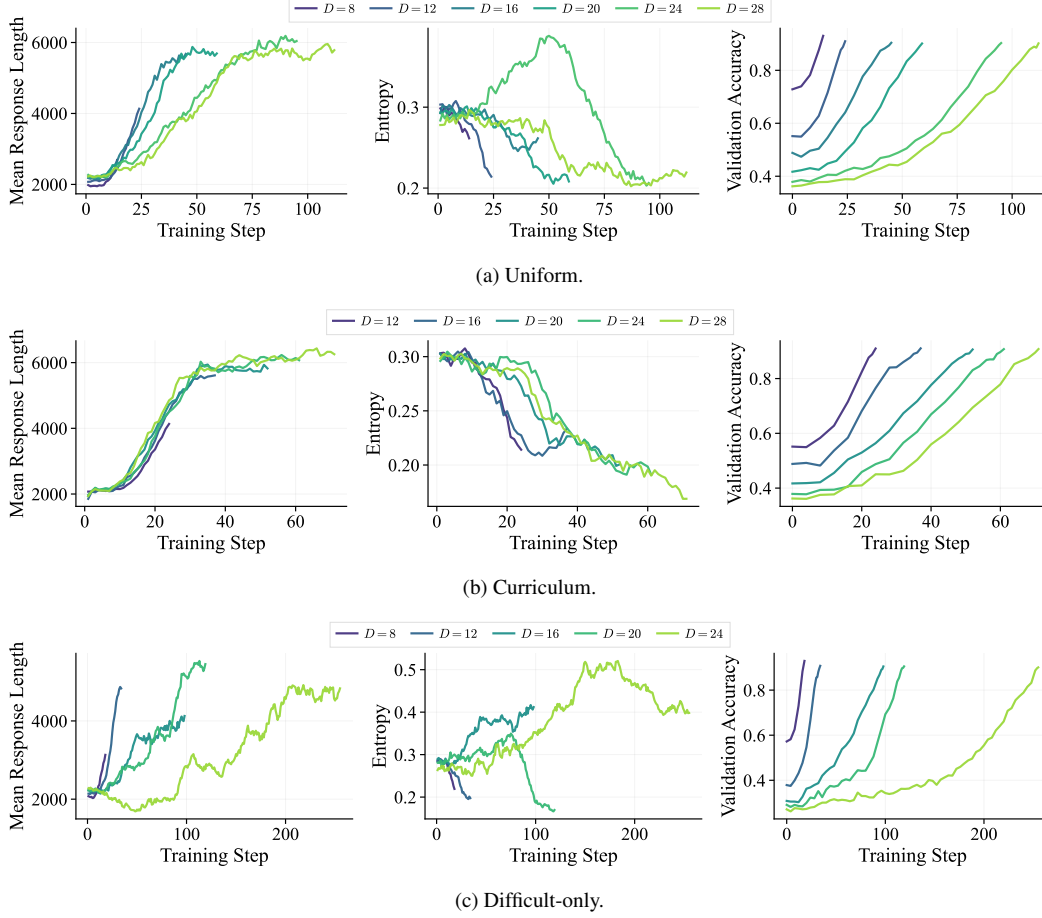


Figure 11: Training trajectories on + *Conjunction* under three data distributions. Each row: response length (left), entropy (middle), validation accuracy (right); legend (training depth D) is shared across the three sub-panels. Note that axis ranges may differ across rows to improve readability.

The scaling behavior at 8B closely mirrors that at 4B. All five expressiveness settings continue to follow a clean power law in proof depth, with $R^2 \geq 0.98$. The fitted exponent γ remains monotonically increasing with logical expressiveness, ranging from 0.99 for *Implication-only* to 2.53 for + *Quantification*, and the overlapping standard-error ranges between + *Conjunction* ($\gamma = 1.52$) and + *Negation* ($\gamma = 1.58$) are also preserved.

Compared with the 4B model, the 8B exponents are systematically smaller across all settings (e.g., 1.52 vs. 1.72 under + *Conjunction*), consistent with the intuition that larger models scale more efficiently with reasoning depth. The relative ordering of the five settings and the qualitative dependence on expressiveness are unchanged, indicating that the scaling phenomena reported in the main text are not artifacts of the 4B model.

As an additional downstream check, the Qwen3-8B model trained at the deepest + *Quantification* setting ($D = 14$) improves the eight-benchmark average from 50.93% to 60.53%, a +9.60 percentage-point gain, suggesting a transfer trend similar to that observed for Qwen3-4B.

J Full Training Dynamics of the Main Experiments

To accompany the training-step scaling results reported in the main text, we visualize the full training trajectories of the five main experimental settings. For each setting, we plot three quantities as a function of the training step: (i) the mean response length, (ii) the actor entropy, and (iii) the validation accuracy (the same quantity used to define the training compute T in the main paper).

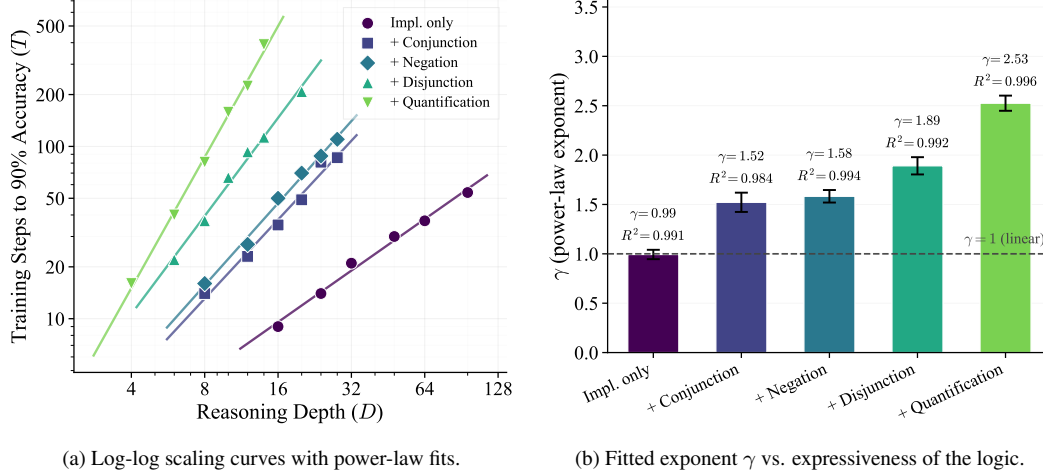


Figure 12: Cross-scale replication on Qwen3-8B. **(a)** Training steps to convergence vs. reasoning depth across five expressiveness levels on log-log axes. Solid lines show power-law fits $T \propto D^\gamma$. **(b)** Fitted γ increases monotonically when expressiveness increases from *Implication-only* ($\gamma=0.99$) to the + *Quantification* setting ($\gamma=2.53$), mirroring the 4B picture in Figure 2 but with systematically smaller exponents. Error bars denote ± 1 standard error of the fitted exponent.

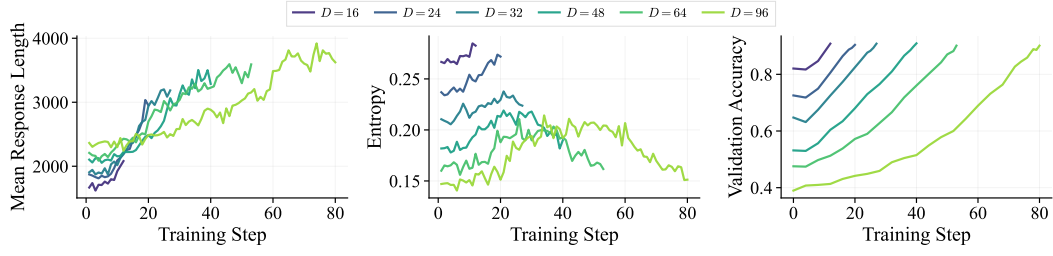
Across the five settings, Figure 13 reveals a consistent set of training dynamics. First, validation accuracy eventually reaches the 0.9 threshold for all trained depths, but deeper problems require systematically more training steps to reach this threshold. These data points are exactly the quantities used to construct the scaling curves in the main text. Second, response length tends to grow during training, especially at larger depths and in more expressive settings, indicating that RL often induces longer reasoning traces as the task requires deeper proof search. Third, actor entropy generally decreases as training progresses, suggesting that the policy gradually concentrates on more reliable reasoning patterns; on harder configurations, this decrease is sometimes preceded by a longer exploratory phase. Together, these trajectories show that the measured scaling behavior is not an artifact of isolated convergence points, but reflects broader changes in accuracy, reasoning length, and policy uncertainty throughout training.

K Evaluation of Frontier LLMs on SCALELOGIC

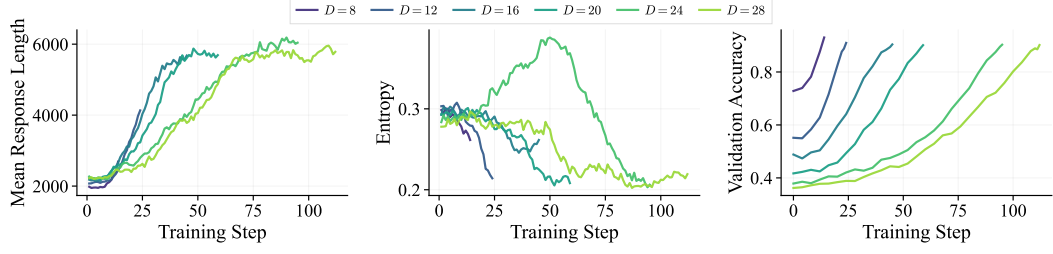
To verify that our synthetic tasks present a genuine reasoning challenge even for frontier models, we evaluate six strong LLMs in the + *Quantification* setting with $B = 4$ across reasoning depths $D \in \{4, 6, \dots, 32\}$. For each depth, we sample 30 tasks and report the number of correct answers. The evaluated models include both non-thinking LLMs (GPT-4o [Hurst et al., 2024] and DeepSeek-V3.1 [Liu et al., 2024]) and LRMs (GPT-5.4-mini [Singh et al., 2025], DeepSeek-R1 [Guo et al., 2025a], Kimi-K2.5 [Team et al., 2026], and Qwen3.5-397B-A17B [Team, 2026]). We use greedy decoding with temperature $T = 0$ whenever temperature control is supported. For GPT-5.4-mini, whose temperature cannot be modified, we use its default setting and set the reasoning effort to “high”. To minimize failures due to truncated outputs, we set the maximum response length to 65,536 tokens.

As shown in Figure 14, increasing reasoning depth leads to clear performance degradation across all evaluated models. Non-reasoning LLMs fail rapidly: GPT-4o and DeepSeek-V3.1 reach near-random accuracy by $D = 8$ and $D = 20$. Although LRMs are more robust, with Qwen3.5-397B-A17B remaining above random even at $D = 32$, they also exhibit marked degradation beyond $D = 12$. These results suggest that the most expressive tasks of SCALELOGIC remain challenging even for frontier reasoning models.

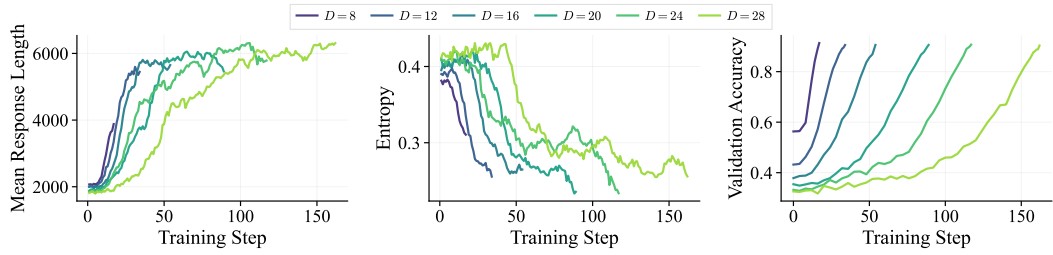
Notably, Figure 5 shows that targeted RL training on SCALELOGIC substantially closes the in-distribution gap between a 4B model and much larger frontier models evaluated zero-shot. At $D = 28$, the RL-trained 4B model maintains 40% accuracy, matched only by Qwen3.5-397B-A17B among the evaluated frontier models. Thus, the comparison highlights distribution-specific gains



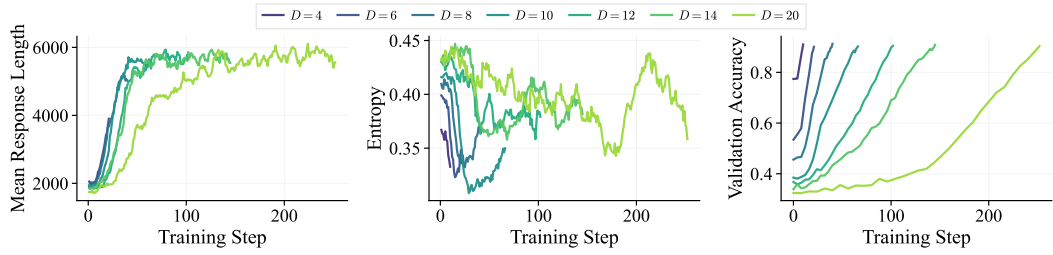
(a) Impl. only



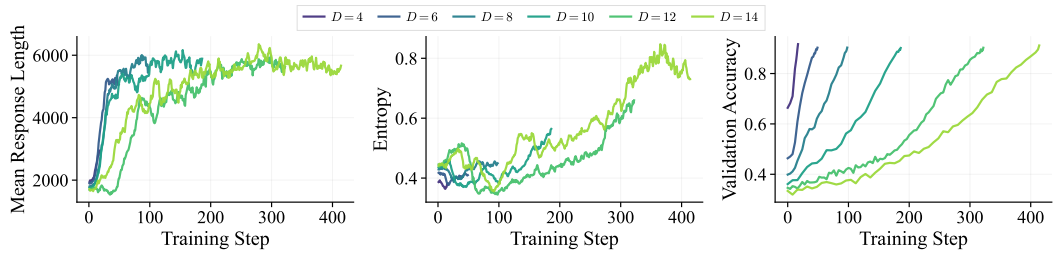
(b) + Conjunction



(c) + Negation



(d) + Disjunction



(e) + Quantification

Figure 13: Training trajectories for all five settings. Each row: response length (left), entropy (middle), validation accuracy (right); legend (training depth D) is shared across the three panels. Note that axis ranges may differ across rows to improve readability.

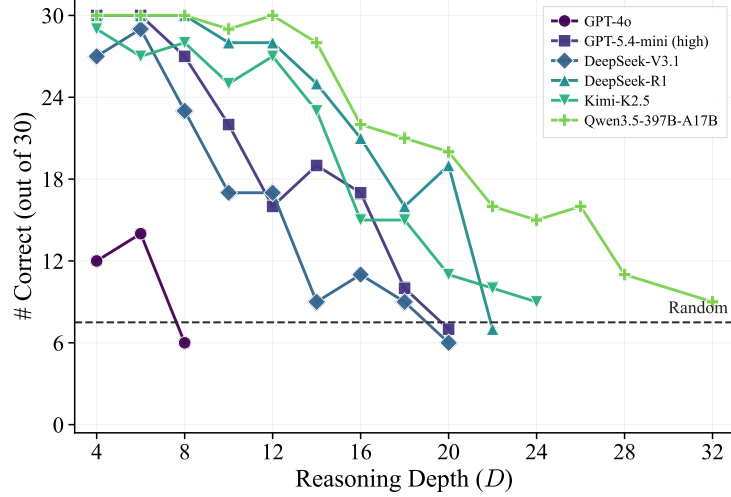


Figure 14: Number of correct answers out of 30 for frontier LLMs on SCALELOGIC (+ *Quantification* setting, $B=4$) as a function of reasoning depth. All models degrade with increasing depth.

from targeted RL, while the continued degradation of all models at larger depths confirms that the depth axis captures a meaningful reasoning challenge.

L Qualitative Examples

Beyond the aggregate accuracy gains in Section 4.3, Figure 15 illustrates how RL post-training changes the model’s reasoning behavior. We compare our model with the base model on MATH-500 #80, a counting problem over a parameterized logarithmic equation whose answer is 501. Our model solves the problem in 7/8 rollouts, whereas the base model solves it in 0/8.

Both models reduce the equation to $x^2 + (4 - k)x + 4 = 0$ and consider the discriminant condition $\Delta = 0$, which only gives $k \in \{0, 8\}$. The base model stops there and answers 2. In contrast, our model identifies a second case: the quadratic may have two real roots, but only one lies in the logarithm domain ($kx > 0, x > -2$). It then splits into $k > 0$ and $k < 0$, uses Vieta’s formulas and the sign of $f(-2)$ to verify the valid-domain cases, and concludes that every negative integer k also works, yielding $500 + 1 = 501$. This example suggests that training on SCALELOGIC encourages a more systematic branching strategy: the model searches for hidden cases, enumerates alternatives, and verifies each branch under explicit constraints.

Question (MATH-500 #80)

Find the number of integer values of k in the closed interval $[-500, 500]$ for which the equation $\log(kx) = 2 \log(x+2)$ has exactly one real solution.

Gold answer: 501

Ours (Qwen3-4B + RL on synthetic logic) 7/8 correct

We are given the equation $\log(kx) = 2 \log(x+2)$. We are to find the number of integer values of $k \in [-500, 500]$ for which this equation has **exactly one real solution**.

Step 1: Understand the domain

The logarithm is only defined for positive arguments, so $kx > 0$ and $x+2 > 0 \Rightarrow x > -2$. Depending on the sign of k :

- If $k > 0$, then $x > 0$.
- If $k < 0$, then $-2 < x < 0$.

If $k = 0$, then $\log(kx) = \log 0$ is undefined, so $k = 0$ is **not allowed**.

Step 2: Solve the equation

$\log(kx) = \log((x+2)^2)$, so if both sides are defined, $kx = (x+2)^2$, giving

$$x^2 + (4-k)x + 4 = 0.$$

For “exactly one real solution” we set the discriminant to zero:

$$\Delta = (4-k)^2 - 16 = 0 \Rightarrow k \in \{0, 8\}.$$

$k = 0$ is invalid; only $k = 8$ remains. *But this seems too few.*

Wait — but maybe there’s more to it. Even if the discriminant is **not zero**, the quadratic might still have **exactly one real solution in the domain**: if the quadratic has two real roots, only one of them might lie in the domain of the original equation. So we need to be more careful.

Step 3: Re-express the problem

The number of real solutions of the original equation equals the number of roots of $x^2 + (4-k)x + 4 = 0$ that lie in the domain. Two cases must be checked:

1. Discriminant = 0, and the unique root is in the domain.
2. Discriminant > 0, and **only one** of the two roots is in the domain.

Case 1: Discriminant is zero $\Rightarrow$ one real solution

$k = 0$: invalid. $k = 8$: $(x-2)^2 = 0 \Rightarrow x = 2$, which lies in the domain $x > 0$. So $k = 8$ is valid. [...]

Case 2: Quadratic has two real solutions, but only one in the domain

By Vieta’s formulas, $x_1 + x_2 = k - 4$ and $x_1 x_2 = 4 > 0$, so both roots share a sign.

Subcase A: $k > 0$.

Both roots are positive, so both lie in the domain $x > 0 \Rightarrow$ **two solutions**, not acceptable. [...] Therefore no $k > 0$ works.

Subcase B: $k < 0$.

Sum $k - 4 < -4$, product $4 > 0$, so both roots are **negative**. The domain is $-2 < x < 0$. We want exactly one root in $(-2, 0)$ and the other ≤ -2 .

Let $f(x) = x^2 + (4-k)x + 4$. Since the parabola opens upward and both roots are negative, the desired configuration is equivalent to $f(-2) < 0$. Compute:

$$f(-2) = 4 - 2(4-k) + 4 = 2k.$$

Hence $f(-2) < 0 \Leftrightarrow k < 0$. So **for every** $k < 0$, the equation has exactly one real solution. [...]

Final Answer.

The valid integers $k \in [-500, 500]$ are:

- All negative integers from -500 to -1 : 500 values.
- Plus $k = 8$ from Case 1.

$500 + 1 = 501$

Base (Qwen3-4B, no RL) 0/8 correct

We are given $\log(kx) = 2 \log(x + 2)$ and we are to find the number of **integer values of k** in $[-500, 500]$ for which the equation has **exactly one real solution**.

Step 1: Understand the equation

Using $\log(a) = \log(b) \Rightarrow a = b$ we obtain

$$kx = (x + 2)^2.$$

[...]

Step 2: Solve the equation

$x^2 + (4 - k)x + 4 = 0$. We want exactly one real solution, which happens when the **discriminant is zero**:

$$\Delta = (4 - k)^2 - 16 = 0 \Rightarrow 4 - k = \pm 4.$$

Solving: $k = 0$ or $k = 8$.

So the values of k for which the equation has exactly one real solution are

$$k = 0 \quad \text{and} \quad k = 8.$$

Step 3: Check if these values are in the interval $[-500, 500]$

Both 0 and 8 are in $[-500, 500]$, so they are valid.

Step 4: Are there any other values of k that could make the equation have exactly one real solution?

We need to make sure that **only** when the discriminant is zero does the equation have exactly one real solution. In general, a quadratic has two real solutions if $\Delta > 0$, one if $\Delta = 0$, none if $\Delta < 0$. So the only values of k are those for which the discriminant is zero, namely $k = 0$ and $k = 8$.

Final Answer.

2

Figure 15: MATH-500 #80: full reasoning trace of our model (top, 7/8 correct) and the base model (bottom, 0/8 correct). Wrong steps in the base model are highlighted in red; “[...]” marks omitted verbose passages.